



NUMIDIA INSTITUTE OF TECHNOLOGY

END OF STUDIES PROJECT REPORT

**Arabic Sign Language (ARSL)
Recognition and Translation into Text**

Authors:

Lakehal Maya Amani
Haddouche Milissa

Supervisors:

Mr. Kaddouri Nassim

*An end-of-studies project report submitted in fulfillment of the
requirements for the degree of Bachelor in Autonomous Systems and
Embedded Ambient and Mobile Software in*

Computer Science

Academic year : 2024– 2025

Acknowledgements

*First and foremost, we express our deepest gratitude to our supervisor, **Mr. Kaddouri Nassim**, for his invaluable guidance, unwavering support, and insightful advice throughout the duration of this project. His expertise and mentorship have been instrumental in shaping our research and academic growth.*

*We extend our sincere thanks to **Mr. Aitameur**, Headmaster of our institution, for providing us with the opportunity and resources necessary to undertake this project.*

Our heartfelt appreciation goes to our families, especially our parents, for their constant encouragement, patience, and unwavering belief in our abilities. Their support has been a cornerstone of our academic journey.

*We are also grateful to our friends **Nour, Mohand and Sohaib**, and others for their assistance, motivation, and camaraderie, which have greatly contributed to the completion of this work.*

*Finally, we would like to thank **Professor Belgacem Haba** for presenting us with this opportunity and for his support throughout our research endeavors.*

Abstract

Sign languages are the primary means of communication for millions with hearing impairment, yet access to qualified interpreters is scarce—particularly in the Arab world, where multiple national and regional variants of Arabic Sign Language (ArSL) coexist and hinder mutual understanding. Recent advances in computer vision and deep learning now enable real-time gesture recognition, offering a scalable alternative to bridge this gap. In this study, we assembled a corpus of raw ArSL video clips and applied frame extraction and preprocessing before training two architectures a **MobileNetV2 + GRU** hybrid and an **Inflated 3D ConvNet (I3D)**. We monitored precision, recall, F1 score, and validation accuracy throughout training. Our best MobileNetV2 + GRU model achieved 96% validation accuracy and 97% test accuracy, with precision and recall both above 95%. These results demonstrate that lightweight, mobile-friendly networks can deliver near-state-of-the-art performance for ArSL recognition. Deploying such a system can reduce communication barriers, minimize misunderstandings, and empower deaf and hard-of-hearing Arabic speakers by providing an always-available bridge where human interpreters are unavailable.

Keywords: Sign language, ArSL, Computer vision, Deep learning, Real-time gesture recognition, MobileNetV2, GRU, I3D, mobile-friendly networks, frames, video clips

Table of Contents

Introduction	1
Chapter 1: Background & Related Work	2
Chapter Introduction	2
1 Background	2
1.1 Definition and Nature of Sign Languages	2
1.1.1 Visual-Gestural vs. Spoken Languages	2
1.1.2 Linguistic Components of Sign Languages	3
1.2 Importance of Sign Languages	3
1.2.1 The Need for Sign Language	3
1.2.2 Sign Language and Connection	4
1.3 Diversity of Sign Languages	4
1.3.1 American Sign Language (ASL)	4
1.3.2 British Sign Language (BSL)	5
1.3.3 Other Sign Languages	5
1.4 Overview of ArSL	5
1.5 Linguistic Features of ArSL	5
1.6 Challenges in ArSL Standardization	6
1.7 Sociocultural Context	6
1.8 Artificial Intelligence	6
1.9 Deep Learning	7
1.10 CNNs	8
1.11 RNNs	8
1.12 GRUs	9
1.13 MobileNetV2	10
1.14 3D CNNs	10

2	Related Works	12
2.1	Video Classification	12
2.1.1	Video-Based Emotion Recognition using CNN-RNN and C3D Hybrid Networks	12
2.1.2	Human Activity Recognition Using LSTM-RNN Deep Neural Network Architecture	13
2.1.3	I3D-LSTM: A New Model for Human Action Recognition	14
2.1.4	I3D-Shufflenet Based Human Action Recognition	16
2.1.5	Real-time detection and monitoring of public littering behavior using deep learning for a sustainable environment	18
2.2	Sign Language Recognition	20
2.2.1	ASL-3DCNN: American Sign Language Recognition Using 3D Convolutional Neural Networks	20
2.2.2	A vision-based deep learning approach for independent-users Arabic sign language interpretation	21
2.2.3	ALGSL89: An Algerian Sign Language Dataset	23
2.2.4	ASLDetect: Arabic sign language detection using ResNet and U-Net like component	25
	Chapter Conclusion	26
	Chapter 2: Methodology	27
	Chapter Introduction	27
1	User Interface and User Experience (UI/UX) Design	27
1.1	User Flow and Navigation	27
1.2	SignBridge Identity	28
1.3	Accessibility and UX Principles	30
1.3.1	Simplicity	30
1.3.2	Consistency	30
1.3.3	Visual Accessibility	30
1.4	Wireframing and Prototyping	30
2	Development of SignBridge	32
2.1	Objective	32

2.2	Functionalities	32
2.3	Technology Stack	32
2.3.1	Frontend (Flutter)	32
2.3.2	Backend	33
2.4	Architecture of SignBridge	35
2.5	Interface of SignBridge	36
2.5.1	Get Started Interface	36
2.5.2	Sign In / Log In Interface	37
2.5.3	Home Interface	37
2.5.4	Recording / Uploading a Video Interface	38
2.5.5	Profile page	39
3	UML Modeling of SignBridge	41
3.1	Use Case Diagram	41
3.2	Activity Diagram	42
4	Deep Learning Framework	43
4.1	Dataset Preparation	43
4.2	Dataset Split	44
4.3	Preprocessing Techniques	45
4.4	Architectures Used	48
4.4.1	MobileNetV2 + GRU	48
4.4.2	I3D	54
	Chapter Conclusion	57
	Chapter 3: Results & Discussion	58
	Chapter Introduction	58
1	Experimental Setup	58
1.1	Hardware Environment	58
1.2	Software Environment	58
1.3	Test Device Specifications	59
2	Training Configuration	60
2.1	Dataset Splits & Preprocessing	60

2.2	Hyperparameters and Callbacks	60
2.2.1	Optimizer and Loss	60
2.2.2	Training Callbacks	61
2.2.3	Other Hyperparameters	61
2.3	Evaluation Metrics	61
2.3.1	Precision	61
2.3.2	Recall	62
2.3.3	F1-Score	62
2.3.4	Confusion Matrix	62
2.4	Quantitative Results	62
2.4.1	LRCN Vs I3D	62
2.4.2	Ablation Studies	68
2.5	Qualitative Results	68
2.6	Discussion	70
2.6.1	Comparison with Existing Work	70
2.6.2	Interpretation of Results	71
2.6.3	Error Analysis	73
2.6.4	Limitations and Future Work	73
	Chapter Conclusion	75
	Conclusion	77
	References	78

List of Figures

1	Branches of AI. [8]	7
2	Deep Learning Network. [10]	7
3	Convolutional Neural Network. [12]	8
4	Feedforward vs Recurrent Neural Networks. [13]	9
5	Gated Recurrent Unit Architecture. [15]	9
6	MobileNetV2 Network Architecture. [17]	10
7	3D CNN Architecture. [19]	11
8	Hybrid CNN-RNN and C3D architecture for emotion recognition. [20].	13
9	Build LSTM Architecture. [21]	14
10	Building the Graph. [21]	14
11	The I3D-LSTM architecture combining 3D CNN for spatial-temporal features and LSTM for sequence modeling. [22]	15
12	The structure of LSTM cell. Input gate, forget gate and output gate are used to forget or remember the history message. [22]	15
13	Channel Shuffle Operation. [23]	16
14	I3D-Shufflenet Architecture. [23]	17
15	Inception Module Diagram of I3D. [23]	17
16	MoViNet architecture. [24]	18
17	MoViNet architecture. [24]	19
18	A sample video sequence from ASL LVD. [25]	20
19	Cascaded CNN architecture. [25]	21
20	The Building Blocks of the CNN Architecture. [26]	22
21	The Building Blocks of the RNN Architecture. [26]	22
22	Performance Comparison of different models on ALGSL89 dataset. [28]	24
23	Detailed Summary of the Autoencoder Model . [28]	24
24	Architecture of proposed model (ASLDetect). [29]	25
25	User-flow of SignBridge .	28
26	Color Palette of SignBridge.	29
27	Fonts of SignBridge.	29
28	Logo of SignBridge.	29
29	The main pages of SignBridge.	31
30	Backend Workflow of SignBridge.	33

31	API Communication between the Flask Server and the Flutter App. . . .	34
32	backend loads the trained model	34
33	Contact Us Interface Using Firebase.	35
34	Get Started Interface of SignBridge.	37
35	Home Interface SignBridge	38
36	Recording / Uploading a Video Interface SignBridge.	39
37	Profile Page of SignBridge.	40
38	Use Case of SignBridge.	41
39	Activity diagram of SignBridge.	42
40	Video Files per Class in ArSL dataset Train Set.	43
41	Video Files per class in arabic-sign-language-dataset.	44
42	arabic-sign-language-dataset Split.	45
43	Frames Extraction Approach.	47
44	Frame Pre-processing Approach.	47
45	The Architecture of The MobileNetV2 with a Sample Input Image. [38] .	49
46	LSTM Vs GRU Architecture . [41]	50
47	MobileNetV2 + GRU Used Architecture	51
48	Structure of I3D (left) and its detailed inception submodule (right). [47]	54
49	I3D Used Architecture	55
50	Confusion matrix for the best-performing model on the validation set. . .	63
51	Training History of LRCN.	63
52	Training History of I3D.	64
53	Metrics of Both Models on Test Set.	64
54	Per Class Metrics of LRCN on Test Set.	65
55	Per Class Metrics of I3D on Test Set.	66
56	Confusion Matrix of LRCN on Test Set.	67
57	Confusion Matrix of I3D on Test Set.	67
58	Frame from important Video File from The Test Set.	69
59	Frame from me Video File from The Test Set.	69
60	Frame from mother Recorded Video File.	69
61	Frame from eat Recorded Video File.	69
62	Best vs. Worst Performing Classes for I3D on the Test Set	72
63	Best vs. Worst Performing Classes for LRCN on the Test Set	72

List of Tables

1	Summary of reviewed video classification studies: dataset, model, and performance.	19
2	Summary of reviewed SL studies: dataset, model, and performance. . . .	26
3	Performance comparison of LRCN and I3D models over 10 and 20 epochs.	65
4	Performance comparison between I3D and LRCN on the test set	65
5	LRCN per-class F1-score extremes on the test set.	66
6	I3D per-class F1-score extremes on the test set.	66
7	Comparison of model complexity and inference speed on the Kaggle P100 GPU.	68
8	Summary of Ablation Findings.	68
9	Comparison of our LRCN performance against the best existing ArSL-20 result.	70

List of Abbreviations

ASL	American Sign Language
ArSL	Arabic Sign Language
BSL	British Sign Language
LSF	French Sign Language
Auslan	Australian Sign Language
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
GRU	Gated Recurrent Unit
C3D	3D Convolutional Neural Network
I3D	Inflated 3D Convolutional Neural Network
VGG16	Visual Geometry Group 16-layer model
SL	Sequence Length
LRCN	Long-term Recurrent Convolutional Network

Introduction

This study tackles the persistent communication barrier between Arabic Sign Language (ArSL) users and the hearing population by leveraging advances in computer vision and deep learning to build a real-time sign recognition and translation system. Sign language is a rich, visual-gestural language combining hand shapes, facial expressions, and body movements; yet, most research and existing tools focus on Western sign languages, leaving ArSL under-resourced despite its vital role within the Arabic-speaking deaf community. Recognizing sign language poses a dual challenge: spatial understanding of hand configurations and temporal modeling of motion over time. Compounding this, there are few large, annotated ArSL datasets, and signing styles vary widely among individuals and regions.

Our aim is to design and implement a lightweight mobile application, SignBridge, that captures video inputs—either live or uploaded—and translates ArSL gestures into written text with high accuracy and low latency. We employ two complementary architectures: a Long-term Recurrent Convolutional Network (LRCN) based on MobileNetV2 plus GRU layers for efficient frame-level feature extraction and sequence modeling, and an Inflated 3D ConvNet (I3D) that processes spatiotemporal features directly from video clips. Preprocessing uniformly samples 25 grayscale frames per clip, applies median-blur noise reduction, histogram equalization, resizing, and padding to ensure consistent input shape and contrast.

Our objectives are three-fold:

- **Technical:** Develop and compare deep learning models for ArSL recognition, optimizing for mobile deployment.
- **Practical:** Provide a user-friendly app enabling deaf and hearing users to communicate seamlessly across social, educational, and professional contexts.
- **Societal:** Raise awareness of accessibility technologies and promote their integration into public services, thereby fostering greater inclusion for individuals with hearing or speech impairments.

Chapter 1: Background & Related Work

Chapter Introduction

This chapter provides the theoretical foundation and explores the existing work relevant to Arabic Sign Language recognition. As well as the chapter presents related work, including recent advances in video-based classification and deep learning techniques applied to many fields, which form the backbone of modern sign language recognition systems .

1 Background

1.1 Definition and Nature of Sign Languages

1.1.1 Visual-Gestural vs. Spoken Languages

Sign languages are complete, natural languages that use visual-manual communication rather than sound. Unlike spoken languages that rely on auditory signals, sign languages convey meaning through:

- Hand movements and shapes
- Facial expressions
- Body posture and orientation
- Use of signing space

These languages have their own:

- Grammar and syntax (different from spoken language word order).
- Phonology (handshape, location, movement, palm orientation, and facial expression combine to form "phonemes").
- Morphology (complex meanings can be expressed in single signs).

1.1.2 Linguistic Components of Sign Languages

1.1.2.1 Handshapes

- Phonemes of sign languages (e.g., flat hand, fist, extended fingers)
- Example: In ASL, the handshape distinguishes signs like "mother" (open hand) vs. "father" (thumb extended)

1.1.2.2 Facial Expressions

- Grammatical markers (e.g., raised eyebrows for yes/no questions in ASL)
- Emotional tone (e.g., squinted eyes for intensity)

1.1.2.3 Body Movements & Orientation

- Direction of movement changes meaning (e.g., "give-to" vs. "give-me" in ASL)
- Shoulder shifts indicate role-switching in narratives

1.1.2.4 Spatial Grammar

- Use of 3D signing space to show relationships (e.g., placing "objects" in space for reference)
- Classifier predicates (e.g., using handshapes to represent vehicles or people)

1.2 Importance of Sign Languages

1.2.1 The Need for Sign Language

The debate as to whether sign language should be taught in schools has been going on for a long time. For example 87,000 people in the UK use BSL as their primary language, and even more use it as a second language. Due to this, I think SL should be taught even if just as an elective course like French is at GCSE.

Whilst sign language is most used by people with hearing impairments, it also has benefits for autistic people. Some autistic people struggle to develop verbal communication and find sign language easier to learn. For children (autistic or not), it can also be also easier to learn sign language than spoken languages. The muscles in a baby's hands actually develop faster than their mouths, meaning that signing is easier for early communication [1].

1.2.2 Sign Language and Connection

Learning BSL helps to build a stronger connection with people. Despite helping you communicate with people who cannot hear, sign language can also help you become a better listener. This is because sign language requires more concentration on the individual speaking.

It requires you look not only at the speaker's hand gestures but also their facial expressions and body movements. By even raising your eyebrows, you can completely change a sentence from a statement to a question. You cannot be looking at your phone or doing other things during your conversation. And this can help you build a stronger connection with those you converse with.

By learning SL, it also helps you learn about Deaf culture and communities. This gives you a better understanding of the challenges that people with hearing impairments face every day. As well as introducing you to the Deaf community, learning sign language makes more communities accessible to Deaf people. Those who are deaf can feel excluded from so many community activities due to the communication barriers [1].

1.3 Diversity of Sign Languages

Contrary to common belief, there isn't a single, universal sign language. Just like spoken languages, sign languages vary significantly from country to country and even within regions. For instance, American Sign Language (ASL) is different from British Sign Language (BSL), despite both countries speaking English. This diversity stems from the organic evolution of sign languages within deaf communities, often influenced by the culture, history, and social interactions of its users [2]. There are more than 300 different Sign Languages used around the world today. Even in countries where the same language is spoken, Sign Language can have many different regional accents that celebrate subtle variations to people's use and understanding of signs [3].

1.3.1 American Sign Language (ASL)

ASL, used predominantly in the United States and parts of Canada, is a complete, natural language that employs signs made with the hands and other movements, including facial expressions and postures of the body. It is not a manual-gestural representation of spoken English, nor is it pantomime. Instead, ASL possesses its own syntax and grammar [2].

1.3.2 British Sign Language (BSL)

BSL is a rich, full-fledged language with its own grammar and lexicon, used by the Deaf community in the United Kingdom. It's distinct from ASL, with different signs and structures. BSL uses a two-handed alphabet, whereas ASL uses a one-handed version [2].

1.3.3 Other Sign Languages

Around the world, there are numerous sign languages, such as Auslan (Australian Sign Language), LSF (French Sign Language), and many others. Each has developed in its own right and serves as a primary means of communication for deaf communities in their respective regions [2].

1.4 Overview of ArSL

Arabic Sign Language (ArSL) is a form of non-verbal communication for Arabic speakers who are deaf or hard of hearing. Like the Arabic language itself, ArSL has many variations based on region or dialect. Until 2004, there was no standardized ArSL across the Arab world, so the Council of Arab Ministers of Social Affairs compiled a common dictionary with vocabulary from various dialects. Today, standardized ArSL appears in the media, much like Modern Standard Arabic, but regional ArSL dialects remain in use day-to-day. Please note that the examples in this infographic may differ from region to region because of varying ArSL dialects [4].

1.5 Linguistic Features of ArSL

Sign language in the Arab World has been recently recognized and documented. Many efforts have been made to establish the sign language used in individual countries, including Jordan, Egypt, Libya, and the Gulf States, by trying to standardize the language and spread it among members of the Deaf community and those concerned. Such efforts produced many sign languages, almost as many as Arabic-speaking countries, yet with the same sign alphabets. This article gives a tentative account of some sign languages in Arabic through reference to their possible evolution, which is believed to be affected by the diglossic situation in Arabic, and by comparing some aspects of certain sign languages (Jordanian, Palestinian, Egyptian, Kuwaiti, and Libyan) for which issues such as primes,

configuration, and movement in addition to other linguistic features are discussed. A contrastive account that depicts the principal differences among Arabic sign languages in general and the spoken language is given [5].

1.6 Challenges in ArSL Standardization

Standardizing ArSL faces challenges due to regional variations and the lack of comprehensive documentation. Efforts to create a unified sign language must consider these dialectal differences to ensure inclusivity and effectiveness. Additionally, the absence of a visualized representation for standard ArSL complicates the development of educational and technological resources [6].

1.7 Sociocultural Context

The sociocultural context of ArSL is deeply rooted in the experiences of Deaf communities within Arab societies. These communities often face social and educational barriers, leading to limited interaction with hearing populations. Understanding the cultural nuances and societal attitudes towards Deaf individuals is essential for developing effective recognition and translation systems that are culturally sensitive and accessible [5].

1.8 Artificial Intelligence

Artificial intelligence (AI) makes it possible for machines to learn from experience, adjust to new inputs and perform human-like tasks. Most AI examples that you hear about today – from chess-playing computers to self-driving cars – rely heavily on deep learning and natural language processing. Using these technologies, computers can be trained to accomplish specific tasks by processing large amounts of data and recognizing patterns in the data. [7] As shown below in Figure 1

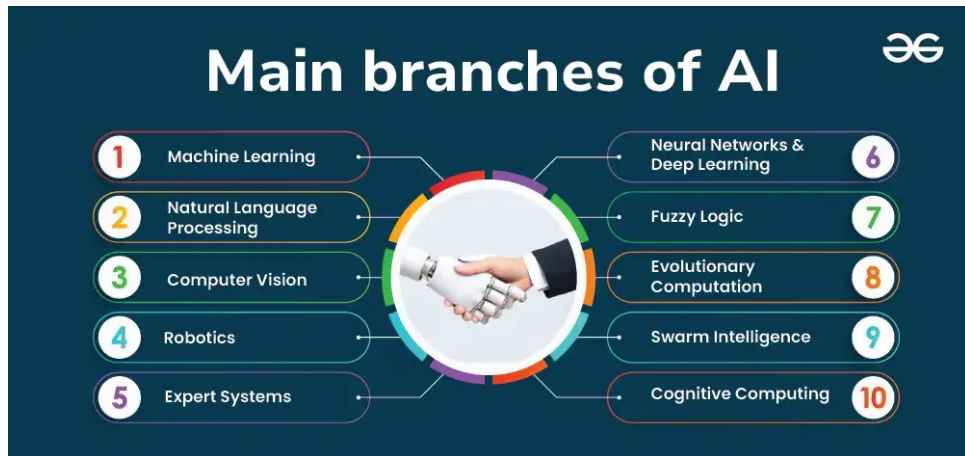


Figure 1: Branches of AI. [8]

1.9 Deep Learning

Deep learning is a type of machine learning that teaches computers to perform tasks by learning from examples, much like humans do. In technical terms, deep learning uses something called "neural networks," which are inspired by the human brain. These networks consist of layers of interconnected nodes that process information. The more layers, the "deeper" the network, allowing it to learn more complex features and perform more sophisticated tasks. [9] As visualized in Figure 2.

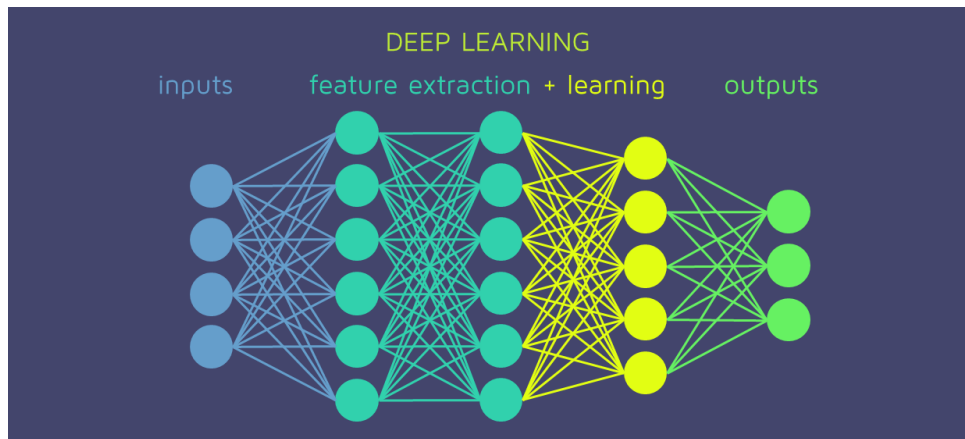


Figure 2: Deep Learning Network. [10]

1.10 CNNs

Convolutional Neural Network (CNN) is an advanced version of artificial neural networks (ANNs), primarily designed to extract features from grid-like matrix datasets. This is particularly useful for visual datasets such as images or videos, where data patterns play a crucial role. CNNs are widely used in computer vision applications due to their effectiveness in processing visual data. [11] Over the years, a variety of CNN architectures have been developed to address different performance and computational trade-offs. These include models like LeNet, AlexNet, VGGNet, GoogLeNet, and ResNet, each introducing key improvements in depth, efficiency, or feature representation. In this study, we focus on *MobileNetV2*, a lightweight yet powerful architecture that is especially well-suited for mobile and embedded applications due to its reduced computational complexity and efficient feature extraction. The Figure 3 below gives an overview about their architectures.

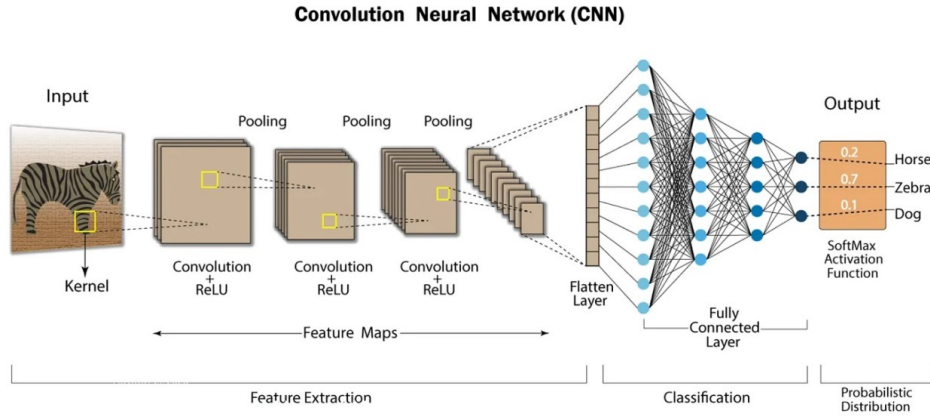


Figure 3: Convolutional Neural Network. [12]

1.11 RNNs

A recurrent neural network or RNN is a deep neural network trained on sequential or time series data to create a machine learning (ML) model that can make sequential predictions or conclusions based on sequential inputs. [13] As shown in the following Figure 4

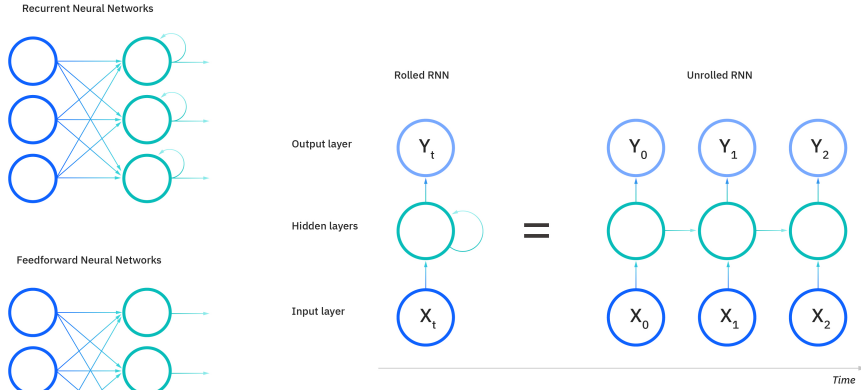


Figure 4: Feedforward vs Recurrent Neural Networks. [13]

1.12 GRUs

A GRU is similar to an LSTM as it also works to address the short-term memory problem of RNN models. Instead of using a “cell state” to regulate information, it uses hidden states, and instead of 3 gates, it has 2: a reset gate and an update gate. Similar to the gates within LSTMs, the reset and update gates control how much and which information to retain. [14] As illustrated in the Figure 5.

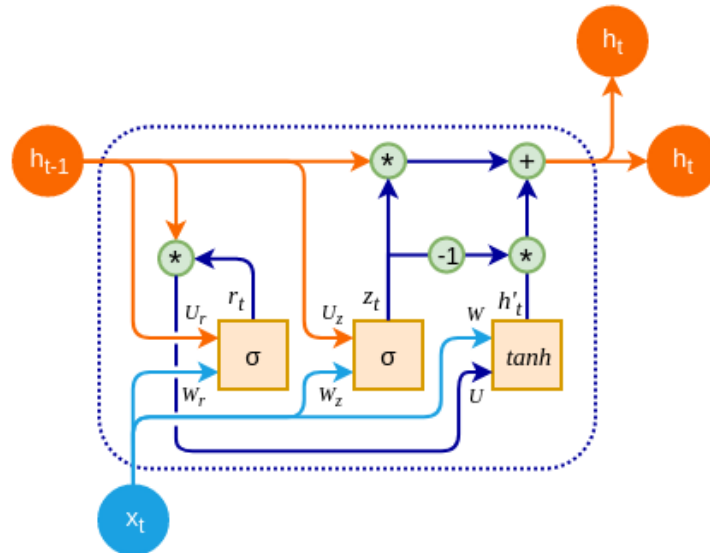


Figure 5: Gated Recurrent Unit Architecture. [15]

1.13 MobileNetV2

MobileNetV2 is a convolutional neural network (CNN) architecture developed by Google, designed for efficient performance on mobile and embedded devices. It enhances the original MobileNet by introducing architectural innovations that balance accuracy with computational efficiency. The main key feature of MobileNetV2 is the use of inverted residuals with linear bottlenecks, where the traditional residual connections are inverted by connecting thin bottleneck layers, and non-linearities are removed from these bottlenecks to preserve information. This design allows for efficient feature extraction and reduced computational cost, making it suitable for deployment on resource-constrained devices. [16] Its architecture is visualized below in Figure 6.

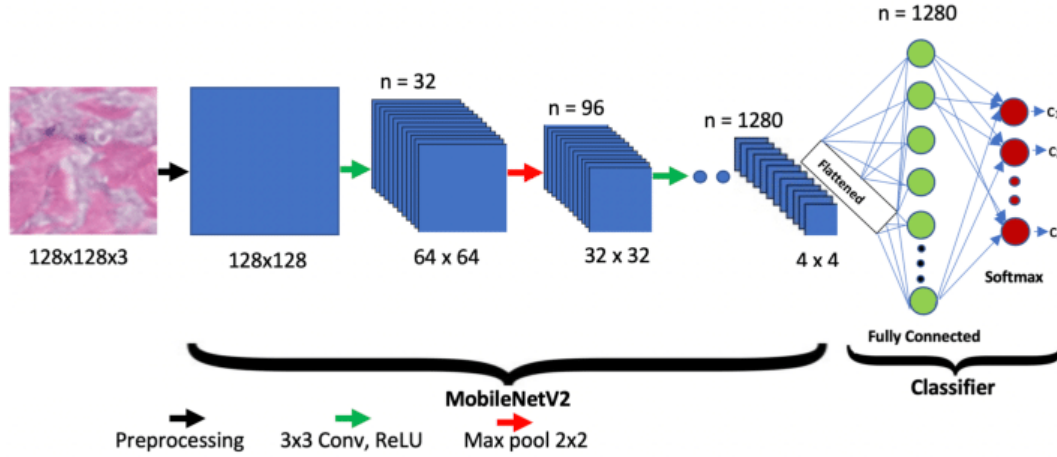


Figure 6: MobileNetV2 Network Architecture. [17]

1.14 3D CNNs

3D Convolutional Neural Networks refer to neural network architectures that extend traditional CNNs by incorporating 3D convolutions, allowing them to process spatio-temporal data such as videos for tasks like human activity recognition. [18] Its architecture is better explained in the Figure 7 below.

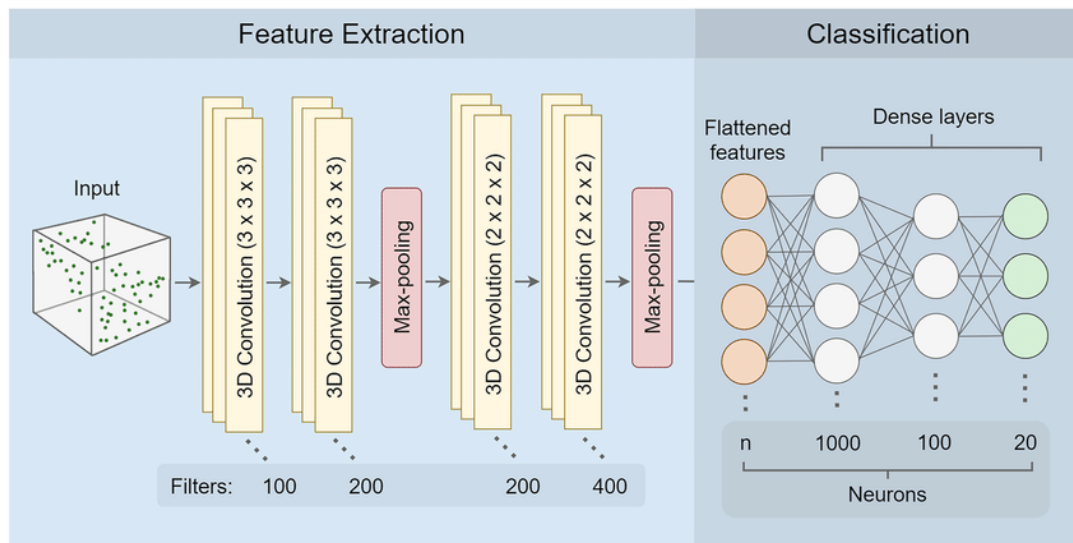


Figure 7: 3D CNN Architecture. [19]

2 Related Works

2.1 Video Classification

2.1.1 Video-Based Emotion Recognition using CNN-RNN and C3D Hybrid Networks

Fan et al. (2016) [20] addressed the challenge of video-based emotion recognition by designing a hybrid system that leverages the complementary strengths of Recurrent Neural Networks (RNNs) and 3D Convolutional Neural Networks (C3D) to capture both frame-level appearance and longer-range spatio-temporal dynamics. The system was evaluated on the AFEW 6.0 dataset, which comprises 1,750 short video clips, each labeled with one of seven basic emotion classes. Faces were detected and aligned in each frame. For the RNN stream, per-frame appearance features were extracted via a VGG16-based CNN fine-tuned on the FER2013 dataset. For the C3D stream, consecutive sequences of 16 aligned face frames were grouped as input clips. An additional audio classifier was trained on the soundtrack of each clip. The RNN module processes the VGG16-extracted face features frame by frame with LSTM units to model temporal dynamics. The C3D module applies 3D convolutions directly to the 16-frame clips to learn joint spatial and short-term temporal features. Outputs from both streams are combined in a late-fusion strategy, and a third stream from the audio classifier is fused with the visual decision scores to produce the final prediction. C3D alone achieved 39.69% accuracy, CNN-RNN alone achieved 45.43%, their visual fusion reached 48.30%, and the full multimodal system (visual + audio) attained 59.02%, outperforming the EmotiW 2015 winner’s 53.80%. Further ensembling of multiple C3D models yielded additional gains, while additional CNN-RNN ensembles provided diminishing returns. The architecture is visualized in Figure 8

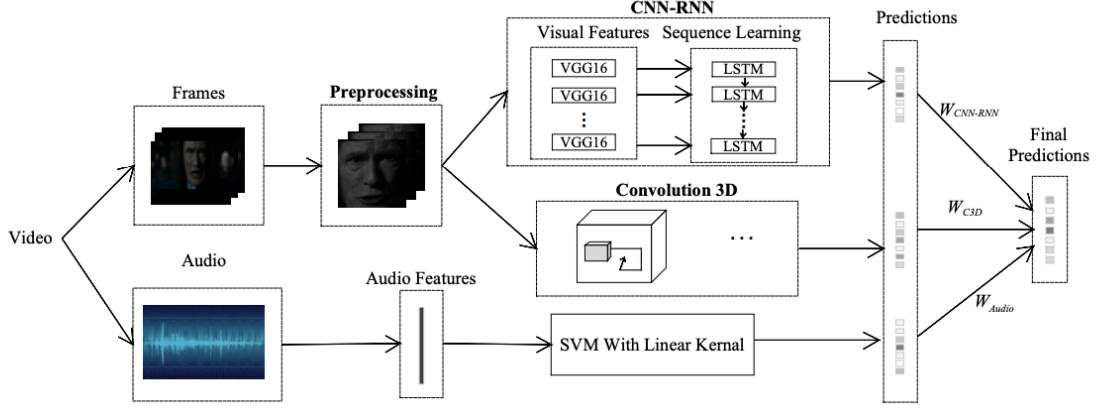


Figure 8: Hybrid CNN-RNN and C3D architecture for emotion recognition. [20].

2.1.2 Human Activity Recognition Using LSTM-RNN Deep Neural Network Architecture

Pienaar and Malekian (2019) [21] tackled the challenge of human activity recognition (HAR) by designing a system that models temporal sequences directly from raw sensor data, eliminating the need for handcrafted features or manual segmentation. Dataset: They evaluated their approach on the WISDM dataset, which provides multi-axis accelerometer readings capturing users' motion across activities like walking, jogging, and standing. Raw accelerometer streams were segmented into continuous time-series windows without any feature engineering or manual slicing; the model learns directly from these raw sequences. A stacked LSTM-RNN architecture was employed, wherein multiple LSTM layers ingest the sensor time-series to capture long-term temporal dependencies and discern patterns corresponding to different activities. The system achieved over 94% classification accuracy and drove training loss below 30% within the first 500 epochs, demonstrating that deep LSTM-based models can deliver real-time, scalable HAR suitable for applications from fitness tracking to safety monitoring. The proposed model steps are shown below in Figures 9 and 10.

```

12. def LSTM_model(inputs):
13.     hidden_bias = tf.Variable(tf.random_normal([HIDDEN_UNITS],
14. mean=1.0))
14.     output_bias = tf.Variable(tf.random_normal([CLASSES]))
15.     hidden_weights = tf.Variable(tf.random_normal([FEATURES, HI
16. DDEN_UNITS]))
16.     output_weights = tf.Variable(tf.random_normal([HIDDEN_UNITS
17. , CLASSES]))
17.     INPUT = tf.transpose(inputs, [1, 0, 2])
18.     INPUT = tf.reshape(INPUT, [-1, FEATURES])
19.     hdn = tf.nn.relu(tf.matmul(INPUT, hidden_weights) + hidden_bias
20. )
20.     hdn = tf.split(hdn, TIME_STEPS, 0)
21.     # 3 LSTM layer stacking
22.     lstm_layers = [tf.nn.rnn_cell.LSTMCell(HIDDEN_UNITS, forget_
23. bias=1.0) for _ in range(3)]
23.     lstm_layers = tf.nn.rnn_cell.MultiRNNCell(lstm_layers)
24.
25.     outputs, _ = tf.contrib.rnn.static_rnn(lstm_layers, hdn, dtype=tf.flo
26. at32)
26.
27.     return tf.matmul(outputs[-1], output_weights) + output_bias

```

Figure 9: Build LSTM Architecture. [21]

```

3. tf.reset_default_graph()
4. INPUT = tf.placeholder(tf.float32, [None, TIME_STEPS, FEATURES
5. ], name="input")
5. LABELS = tf.placeholder(tf.float32, [None, CLASSES])
6. prediction = LSTM_model(INPUT)
7. predict_softmax = tf.nn.softmax(prediction, name="y_")
8.
9. l2_regularization = 0.0015 * sum(tf.nn.l2_loss(trainable_var) for traina
10. ble_var in tf.trainable_variables())
10.
11. cost = tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits(logits=
12. prediction, labels = LABELS)) + l2_regularization
12.
13. optimizer = tf.train.AdamOptimizer(learning_rate=0.0025).minimize(c
14. ost)
14. correct_prediction = tf.equal(tf.argmax(predict_softmax, 1), tf.argmax
15. x(LABELS, 1))
15. accuracy = tf.reduce_mean(tf.cast(correct_prediction, dtype=tf.float3
16. 2))

```

Figure 10: Building the Graph. [21]

2.1.3 I3D-LSTM: A New Model for Human Action Recognition

In their 2019 study, Wang et al. [22] address the challenge of accurately recognizing human actions in videos by proposing a hybrid model that effectively captures both short-term spatial-temporal patterns and long-term temporal dependencies. To support this, they utilize the large-scale Kinetics dataset, which contains 400 action classes, for pretraining. The evaluation is conducted on the UCF-101 dataset using a 9,537-video training split. For preprocessing, they opt for video-specific pretraining on Kinetics rather than traditional image-based datasets like ImageNet, allowing the model to better learn motion-aware features. The proposed I3D-LSTM model integrates a two-stage architecture: I3D for low-level spatiotemporal feature extraction, and LSTM networks for high-level temporal sequence modeling. I3D extends 2D convolutions and pooling operations into 3D to capture spatial and short-term motion cues, while the LSTM component uses gating mechanisms (input, forget, and output gates) to effectively model long-term dependencies and mitigate vanishing gradient issues. This end-to-end approach eliminates the need for handcrafted features or optical flow. The model achieves 95.1% accuracy on UCF-101, surpassing standalone I3D 94.3% and other leading approaches like C3D, Two-Stream networks, and LRCN. The study highlights the benefits of combining 3D CNNs and LSTMs, though it also notes potential overfitting on smaller datasets and recommends future testing on broader benchmarks. The Figure 11 illustrate the architecture of their model while the 12 figure shows the structure of LSTM cell.

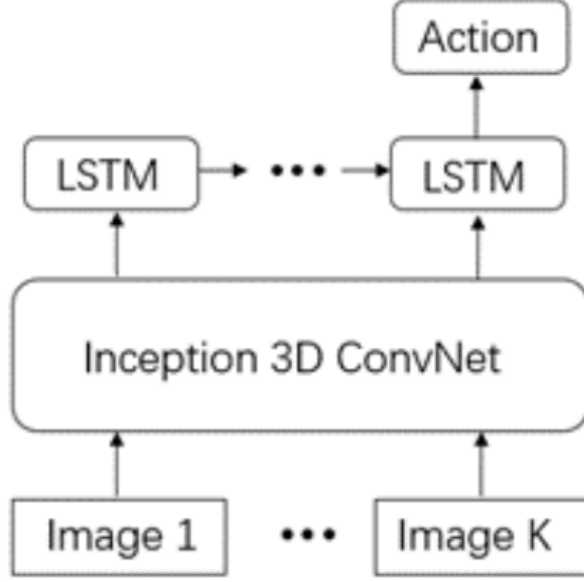


Figure 11: The I3D-LSTM architecture combining 3D CNN for spatial-temporal features and LSTM for sequence modeling. [22]

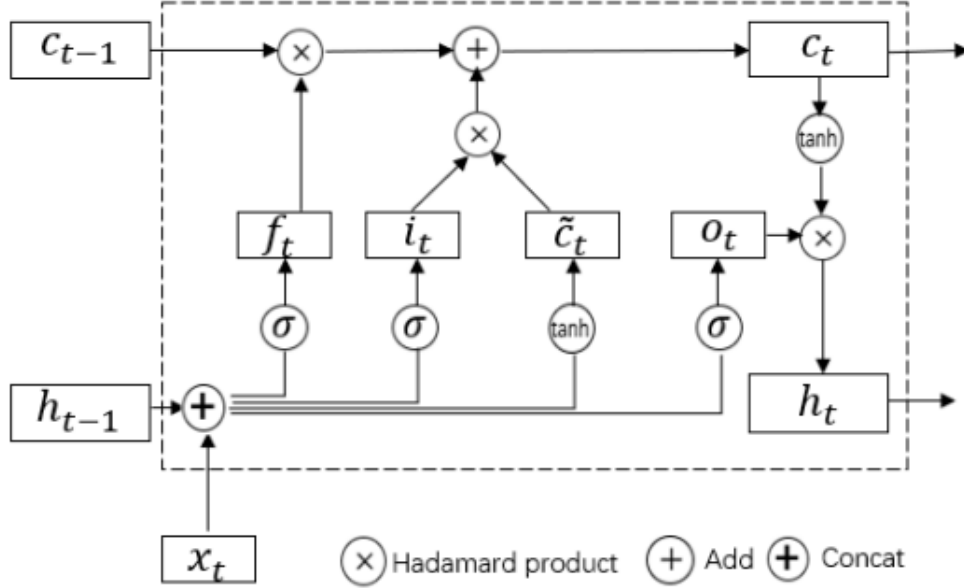


Figure 12: The structure of LSTM cell. Input gate, forget gate and output gate are used to forget or remember the history message. [22]

2.1.4 I3D-Shufflenet Based Human Action Recognition

Li et al. (2020) [23] addressed the need for an efficient yet accurate video-based human action recognition model by combining the representational power of the Inflated 3D ConvNet (I3D) with the lightweight design of ShuffleNet. The model was evaluated on the UCF101 benchmark, which contains 13,320 realistic video clips spanning 101 action categories. To bolster motion feature extraction, Histogram of Oriented Gradients (HOG) descriptors were computed on video frames prior to model input; no manual segmentation beyond standard clip extraction was employed. The proposed I3D-ShuffleNet replaces each original 5×5 3D convolution in I3D with two consecutive 3×3 convolutions, integrates channel-shuffle operations from ShuffleNet to improve information flow and reduce parameters, and leverages Class Activation Mapping (CAM) to visualize spatio-temporal attention during inference. Training was performed using the Adam optimizer with a learning rate of 0.0001. On UCF101, I3D-ShuffleNet achieved 96.4% classification accuracy on par with or better than standard I3D while reducing overall training time by 15.3%. The Figures 13, 14, 15 below give an overview about their approach.

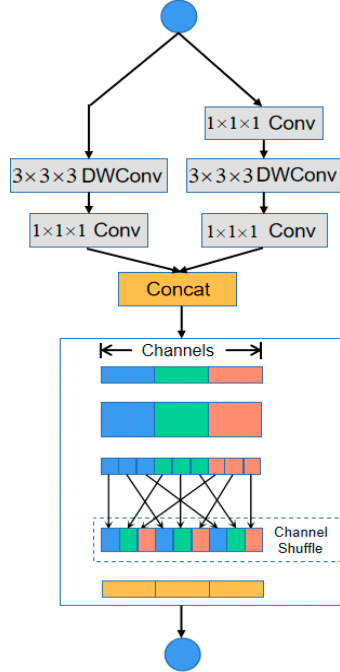


Figure 13: Channel Shuffle Operation. [23]

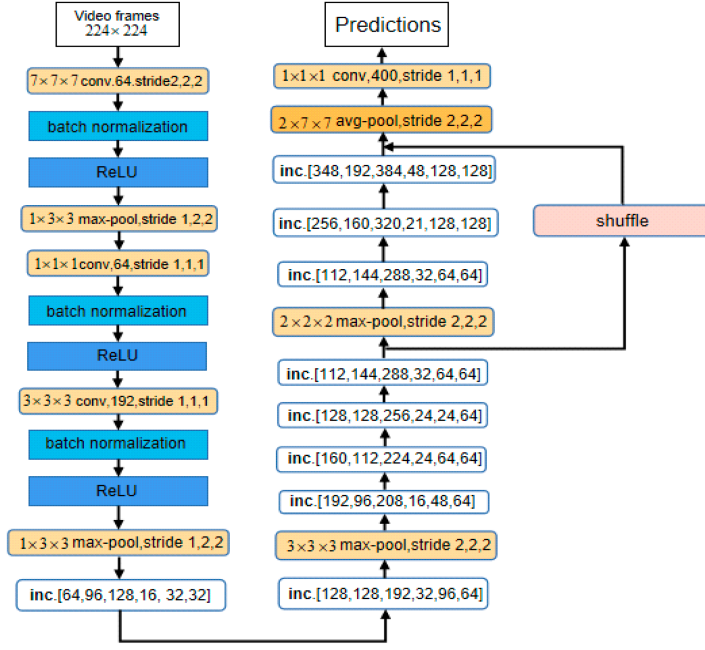


Figure 14: I3D-Shufflenet Architecture. [23]

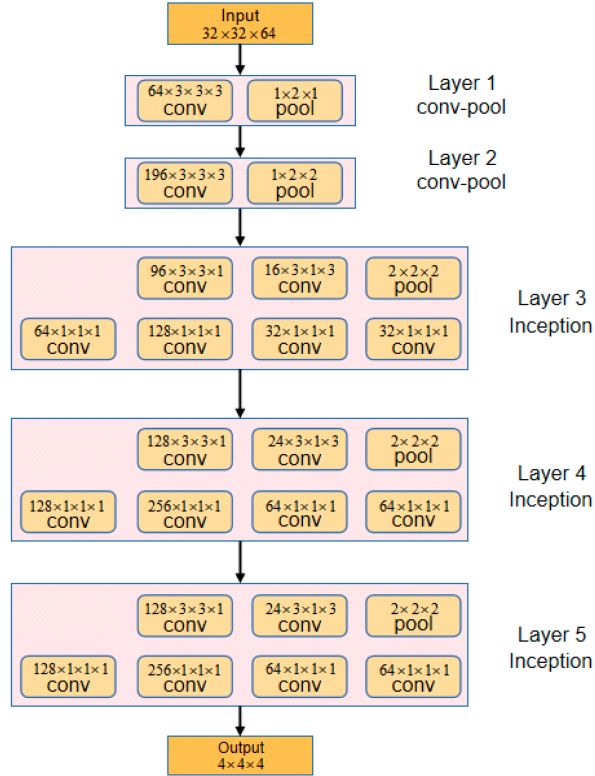


Figure 15: Inception Module Diagram of I3D. [23]

2.1.5 Real-time detection and monitoring of public littering behavior using deep learning for a sustainable environment

Alharbi et al. (2025) [24] addressed the need for a real-time public littering detection system by developing SAWN—a Surveillance and Waste Notification framework that integrates deep learning behavioral classification and object detection to identify pedestrian and vehicle littering in surveillance footage. Because no public datasets existed for this task, they constructed a custom dataset of 276 simulated littering videos, split into 70% training, 15% validation, and 15% testing; all videos were standardized to 1080p HD at a 16:9 aspect ratio with muted audio. To enhance generalization, training and validation videos underwent a variety of augmentations—including horizontal and vertical flips, multiplicative brightness and contrast adjustments, and Gaussian blur—often applied in combination to increase diversity and robustness. The core model is MoViNet (Mobile Video Networks), a lightweight, memory-efficient 3D CNN tailored for real-time video analysis; it was first trained on a multi-class littering classification task but later simplified to a binary classifier distinguishing pedestrian versus vehicle littering, with further performance gains achieved through model ensembling and frame-rate reduction. The final binary MoViNet model achieved 99.5% accuracy on both training and testing sets, with an F1-score of 98.35%, Precision of 98.9%, and Recall of 97.8%, outperforming traditional approaches like OpenPose or Haar-cascade detection in real-world conditions. Used architecture is visualized below in Figures 16 and 17.

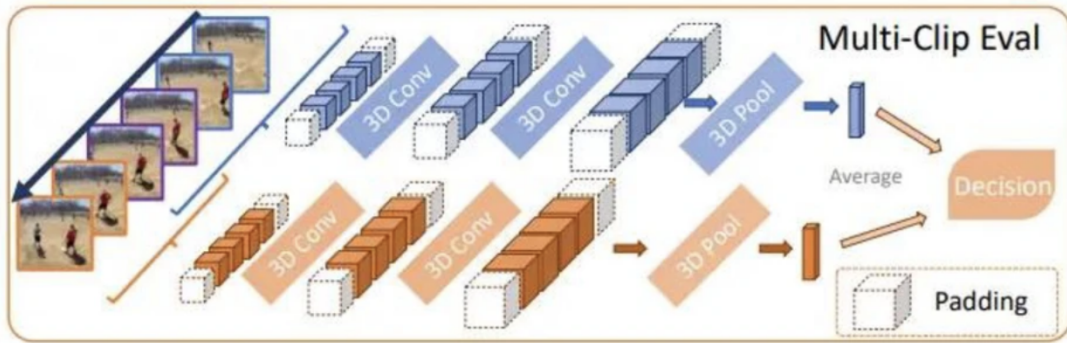


Figure 16: MoViNet architecture. [24]

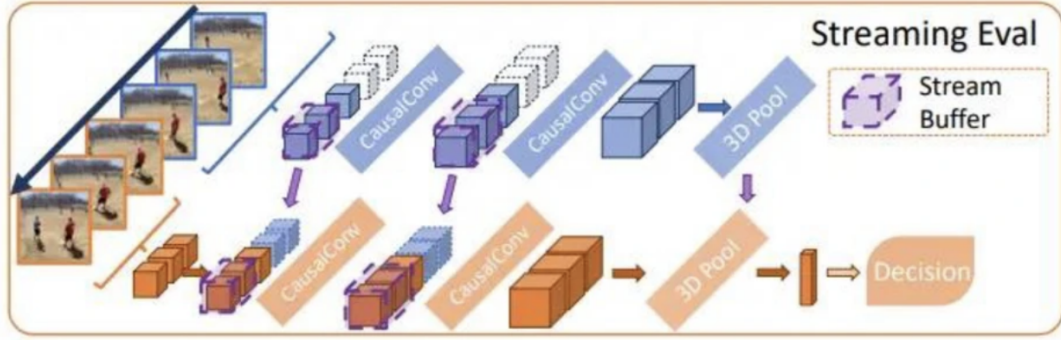


Figure 17: MoViNet architecture. [24]

Works summary

Study (Year)	Dataset	Used Model	Results
Video-Based Emotion Recognition (2016)	AFEW 6.0 (1,750 clips, 7 classes)	VGG16-LSTM (RNN) + C3D + audio (late fusion)	Full multimodal: 59.02% accuracy
Human Activity Recognition (2019)	Kinetics video dataset	I3D-LSTM	95.1% accuracy on UCF-101
Human Activity Recognition (2019)	WISDM (multi-axis accelerometer)	Stacked LSTM-RNN	Over 94% accuracy; loss below 30% within first 500 epochs
Human Action Recognition (2020)	UCF101 (13,320 clips, 101 classes)	I3D-ShuffleNet (inflated 3D + channel shuffle)	96.4% accuracy
Monitoring of Public Littering (2025)	Custom (276 HD videos)	MoViNet (binary pedestrian/vehicle littering)	99.5% accuracy

Table 1: Summary of reviewed video classification studies: dataset, model, and performance.

2.2 Sign Language Recognition

2.2.1 ASL-3DCNN: American Sign Language Recognition Using 3D Convolutional Neural Networks

Kumar and Sharma (2021) [25] address the problem of dynamic American Sign Language (ASL) recognition by proposing a 3D-CNN-based model capable of capturing spatiotemporal features across varying viewpoints. They utilize the Boston ASL Lexicon Video Dataset (LVD) 18, which comprises over 3,300 video samples of 100 ASL words signed by six individuals from four distinct camera angles. In preprocessing, the videos are converted to grayscale, denoised with median filtering, normalized through histogram equalization, resized to 512×384 pixels, and temporally downsampled to 25 frames for uniform input length. The model architecture involves four cascaded 3D convolutional networks—one for each viewpoint—followed by fully connected layers and softmax classification as shown in the figure 19. Trained using the Adam optimizer, the system achieves strong results, including 96% precision, 97.1% recall, and 96.4% F1-score, with real-time performance at 0.19 seconds per frame. The approach effectively handles signer and environmental variability, setting a new benchmark for multi-view dynamic ASL recognition.

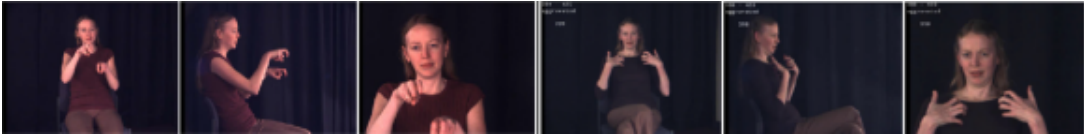


Figure 18: A sample video sequence from ASL LVD. [25]

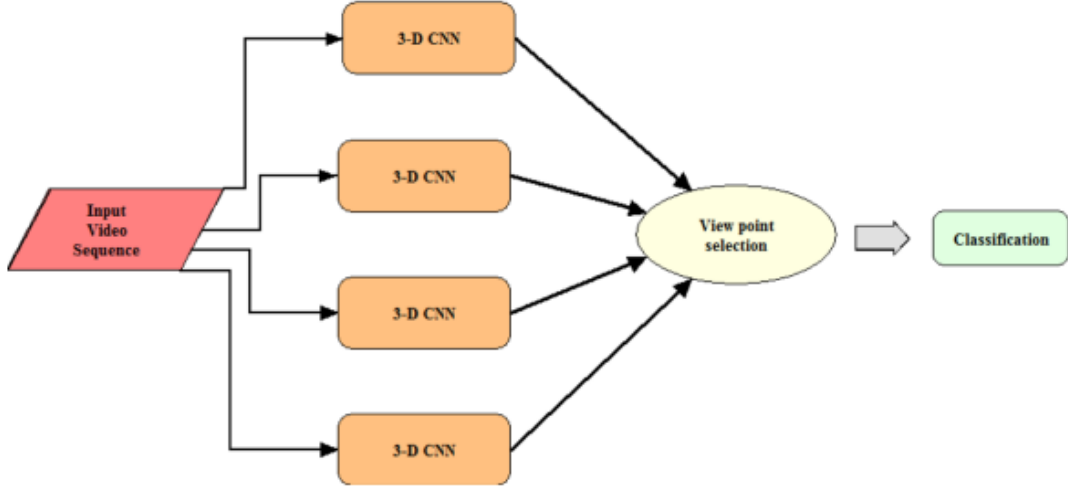


Figure 19: Cascaded CNN architecture. [25]

2.2.2 A vision-based deep learning approach for independent-users Arabic sign language interpretation

Balaha et al. (2023) [26] tackle user-independent isolated-word Arabic sign language recognition by proposing both a new video dataset and a hybrid CNN–RNN model to accurately classify sign words recorded under varying, tool-free conditions. They collected 8,467 smartphone-recorded video clips of 20 distinct Arabic sign words, performed at least five times each by 72 volunteers [27]. Videos are processed through a three-stage pipeline: resize frames and convert to grayscale, apply successive-frame differencing with adaptive thresholding to isolate hand motion, and temporally downsample to a fixed 30-frame sequence by grouping each adjacent triplet of frames into one composite frame. Two parallel CNN streams each comprising four convolution-pooling-dropout blocks with global average pooling extract 512-dimensional spatial feature vectors per frame. These are concatenated into a 30×512 sequence fed into an RNN of five stacked bidirectional LSTM layers (512 units each, with dropout), capped by a 20-way SoftMax classifier. On their Arabic sign dataset, the model achieved $\approx 98\%$ top-1 accuracy on validation and 92% on a held-out test split. It also generalized to UCF-101, yielding 93.4% top-1 and 98.8% top-5 accuracy on that benchmark. The Figures 20, 21 give a better overview about their approach.

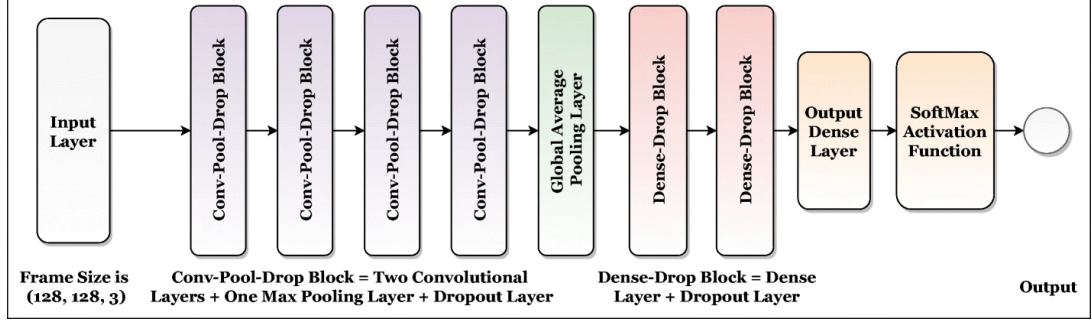


Figure 20: The Building Blocks of the CNN Architecture. [26]

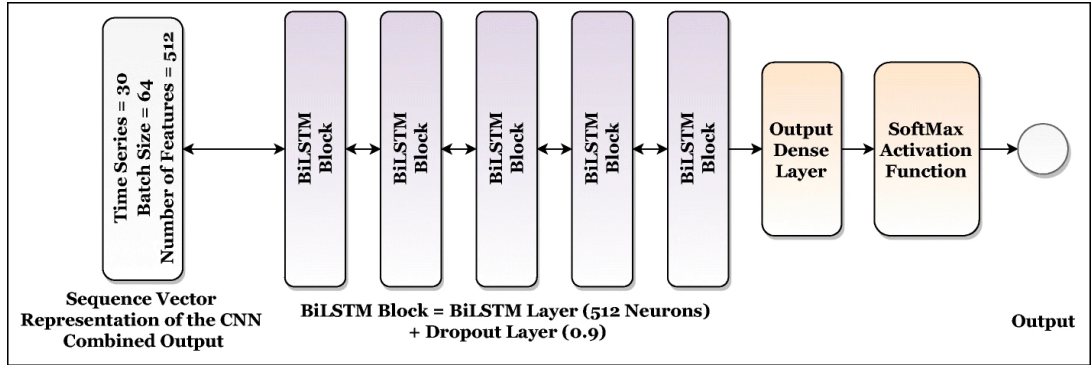


Figure 21: The Building Blocks of the RNN Architecture. [26]

2.2.3 ALGSL89: An Algerian Sign Language Dataset

Problem: Kheldoun et al. (2023) [28] addressed the lack of a large-scale Algerian Sign Language (ALGSL) resource by introducing the first comprehensive video dataset for isolated-sign recognition and benchmarking multiple deep-learning architectures on it. ALGSL89 comprises 4,885 videos covering 89 distinct isolated signs. Each sign was performed five times by 10 non-expert volunteers plus one deaf signer, recorded at 4K/30 fps against a uniform background to balance consistency with natural variation. Videos are first screened for blur using a Laplacian variance test. Non-blurred frames are then fed into a hand-segmentation module that extracts each hand’s bounding box and 21 landmark points per frame. These pose and appearance features are assembled into a fixed-length sequence for model input. The authors implemented and compared four architectures: **2D CNN** for per-frame spatial feature learning, **LSTM** to model temporal dynamics of landmark sequences, **Hybrid CNN–LSTM** combining spatial and temporal streams, and **Autoencoder with attention** that integrates CNN-extracted features, LSTM dynamics, and an attention mechanism to focus on critical frame regions. They further packaged the best-performing Autoencoder model into SignAtlas, a real-time recognition tool that processes webcam or uploaded video to output predicted signs as text and speech. The Autoencoder with attention outperformed the other architectures, achieving the highest classification accuracy and lowest loss on ALGSL89. When deployed in the SignAtlas system, it enables reliable, real-time Algerian Sign Language recognition. Built architecture is shown in Figure 23

Model	Accuracy (%)	Loss	Validation Acc (%)	Validation Loss
LSTM	92.28%	0.2259	60.56%	2.3433
2DCNN-LSTM	71.48%	0.9300	65.00%	0.9395
ConvLSTM	80.48%	0.6930	73.48%	0.9395
Autoencoder	100%	0.00005	98.99%	0.0015

Figure 22: Performance Comparison of different models on ALGSL89 dataset. [28]

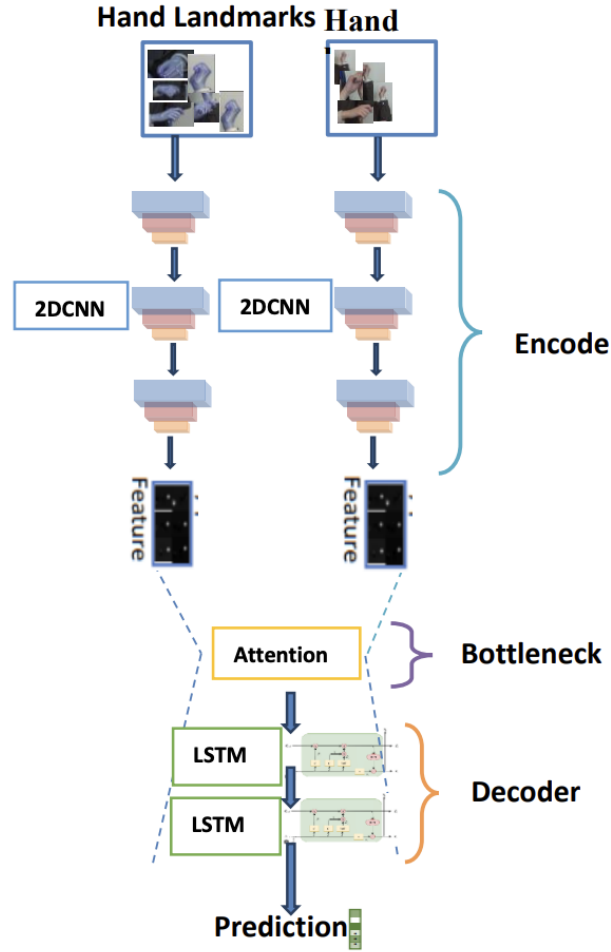


Figure 23: Detailed Summary of the Autoencoder Model . [28]

2.2.4 ASLDetect: Arabic sign language detection using ResNet and U-Net like component

Alasmari and Asiri (2025) [29] addressed the challenge of recognizing Arabic Sign Language (ArSL) in real-world settings, where background noise, lighting variations, and hand occlusions hinder the effectiveness of assistive and educational systems. They evaluated ASLDetect on two ArSL video collections:

- **ArASL2018:** captured against plain backgrounds for controlled gesture recognition, covered 32 classes of Arabic sign language alphabets.
- **ArSL21L:** recorded in complex, diverse environments to test model robustness, comprised 32 classes representing Arabic sign letters.

Input frames were resized to 128×128 pixels, normalized using dataset-specific mean and standard deviation, and subjected to selective augmentations (random flips, brightness/contrast adjustments) to improve resilience across varying conditions. ASLDetect combines a ResNet34 backbone for deep feature extraction with a U-Net–inspired segmentation module to isolate hand gestures from cluttered backgrounds. Training is performed in two stages: initial training on ArASL2018, followed by transfer learning and fine-tuning on ArSL21L. On ArASL2018, ASLDetect achieved 99.35% accuracy surpassing ResNet34 (99.08%), T-SignSys (97.92%), and UrSL-CNN (98%). After transfer learning on ArSL21L, it reached 86.84% accuracy outperforming ResNet34 (82.50%), T-SignSys (58.98%), and UrSL-CNN (49%).

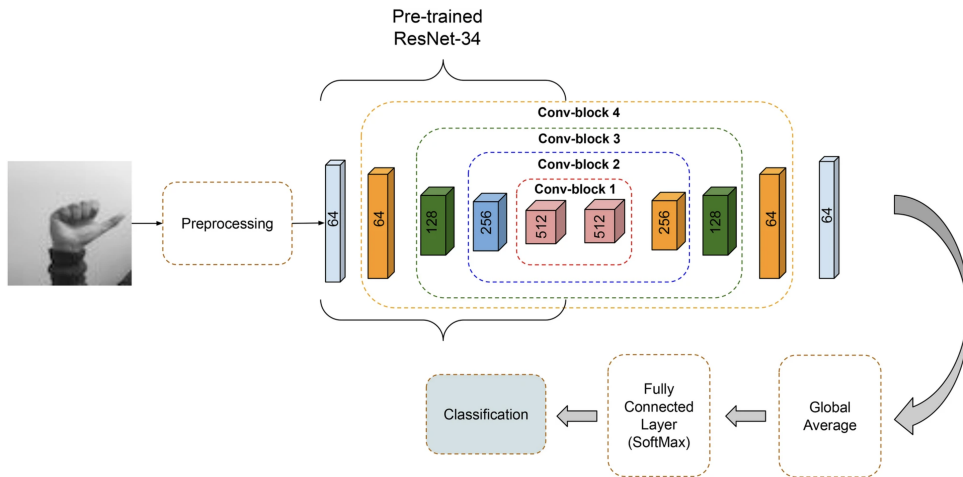


Figure 24: Architecture of proposed model (ASLDetect). [29]

Works summary

Study (Year)	Dataset	Used Model	Results
American sign language recognition (2021)	Boston ASL Lexicon Video	3D-CNNs	96.4% F1-score
Arabic sign language interpretation (2023)	Custom Arabic Sign Language	hybrid CNN–RNN	92% accuracy
An Algerian Sign Language Dataset (2023)	ALGSL89	Autoencoder	98.99% validation accuracy
ASLDetect (2025)	ArASL2018 & ArSL21L	ResNet & U-Net	99.35% accuracy & 86.84% accuracy

Table 2: Summary of reviewed SL studies: dataset, model, and performance.

Chapter Conclusion

In this chapter, we introduced sign languages as rich visual-gestural systems distinct from speech and surveyed their global diversity, with a focus on Arabic Sign Language (ArSL) and its linguistic, sociocultural, and standardization challenges. We then reviewed key AI techniques—CNNs, RNNs/GRUs, MobileNetV2, and 3D CNNs—highlighting their roles in gesture recognition. Finally, a critical appraisal of recent video-based and ArSL-specific studies identified both successful hybrid architectures and resource gaps, motivating our methodology in Chapter 2.

Chapter 2: Methodology

Chapter Introduction

This chapter serves as a comprehensive blueprint detailing the systematic approach undertaken to achieve the research objectives. It encompasses both the technical framework and the application development aspects integral to the project. The application development section delves into the user interface design, the technologies utilized, the functionalities offered, and the software engineering diagrams such as use case and activity diagrams that underpin the system's development. Subsequently, the technical segment elucidates the conception and design of the system, the characteristics and preparation of the dataset, the preprocessing techniques employed, and the architectural choices, including the implementation of pre-trained models and the proposed custom architecture.

1 User Interface and User Experience (UI/UX) Design

1.1 User Flow and Navigation

Before proceeding with the implementation of the application's UI/UX, it was essential to define the user journey when interacting with our system. We focused on the core functionalities that form the foundation of the application, including the option for users to either record live video or upload a pre-recorded video from their gallery to proceed to the sign language recognition phase. Additionally, the application offers a dedicated section where users—both Deaf and hearing—can explore and learn commonly used sign language gestures. To enhance support and accessibility, users also have the ability to contact the administrator, either by sending a direct message or initiating a call using the provided phone numbers. The figure 25 illustrates the flow when interacting with our SignBridge mobile application .

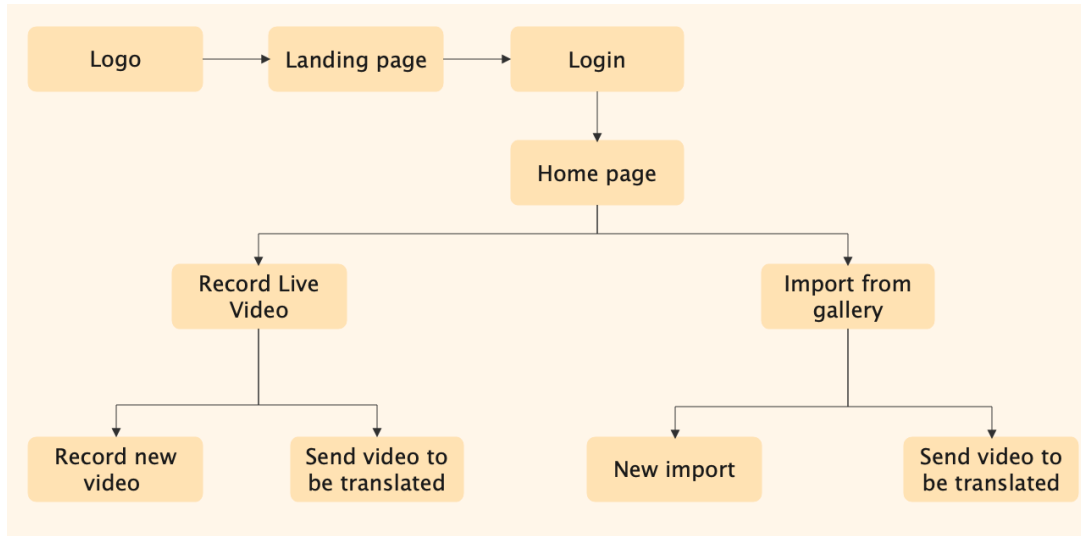


Figure 25: User-flow of SignBridge .

1.2 SignBridge Identity

In the initial stages of designing our application’s UI/UX, we prioritized selecting a color palette that resonates with the Deaf community. Research indicates that colors like green, blue, and orange are particularly engaging for this audience. Drawing inspiration from interfaces within the Figma community, we curated a palette that not only aligns with these preferences but also ensures accessibility and visual appeal. The incorporation of orange, for instance, adds vibrancy and warmth, enhancing user engagement while maintaining clarity and readability. For our application’s typography, we selected two complementary typefaces: Montserrat and BR Hendrix. Montserrat, a geometric sans-serif font , its clean lines and modern aesthetics enhance readability, making it ideal for body text and general content. BR Hendrix, on the other hand, is a neo-grotesque font family characterized by its precise design and geometric flavor. It offers a contemporary look with advanced typographic features, making it suitable for headings and emphasis. Together, these fonts provide a balanced and accessible visual hierarchy, aligning with our goal of creating an inclusive and user-friendly interface for the Deaf community as shown in the 3 figures [26](#), [27](#) and [28](#) .

COLOR PALETTE

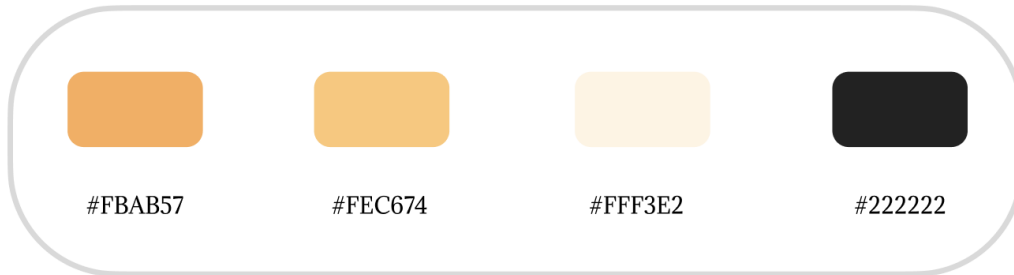


Figure 26: Color Palette of SignBridge.

Fonts

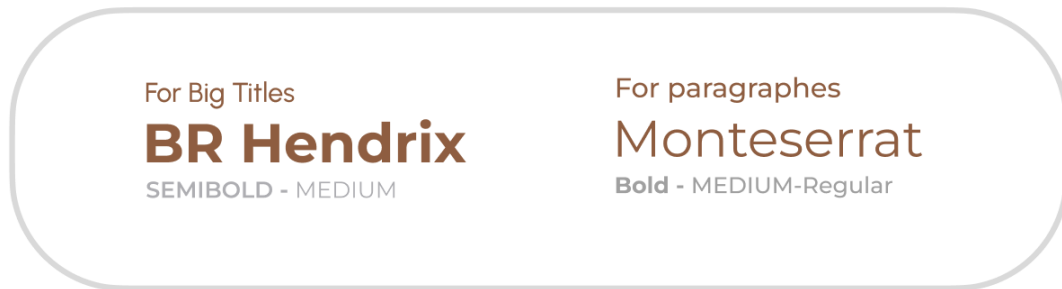


Figure 27: Fonts of SignBridge.

Logo



Figure 28: Logo of SignBridge.

1.3 Accessibility and UX Principles

In designing the SignBridge application, we prioritized accessibility and user experience (UX) principles tailored to the Deaf community. This approach ensures that the application is both inclusive and user-friendly.

1.3.1 Simplicity

We adopted a clean and straightforward design to minimize cognitive load. By avoiding clutter and focusing on essential elements, users can navigate the application with ease.

1.3.2 Consistency

Uniformity in design elements such as buttons, icons, and typography was maintained throughout the application. This consistency aids in predictability and enhances the overall user experience.

1.3.3 Visual Accessibility

Given the reliance on visual cues by Deaf users, we ensured that information is conveyed through clear visuals. This includes the use of icons, images, and readable text to communicate effectively without relying on audio.

1.4 Wireframing and Prototyping

We utilized Figma [30] to create wireframes, providing a clear blueprint of SignBridge's structure and user flow. This step ensured alignment with user needs and streamlined the design process. As demonstrated below in 29.

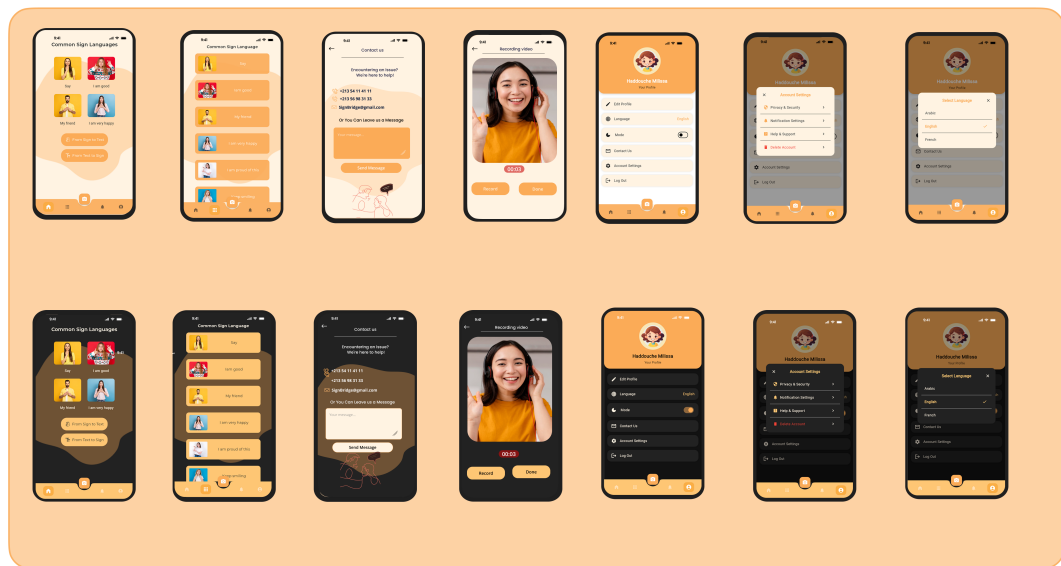
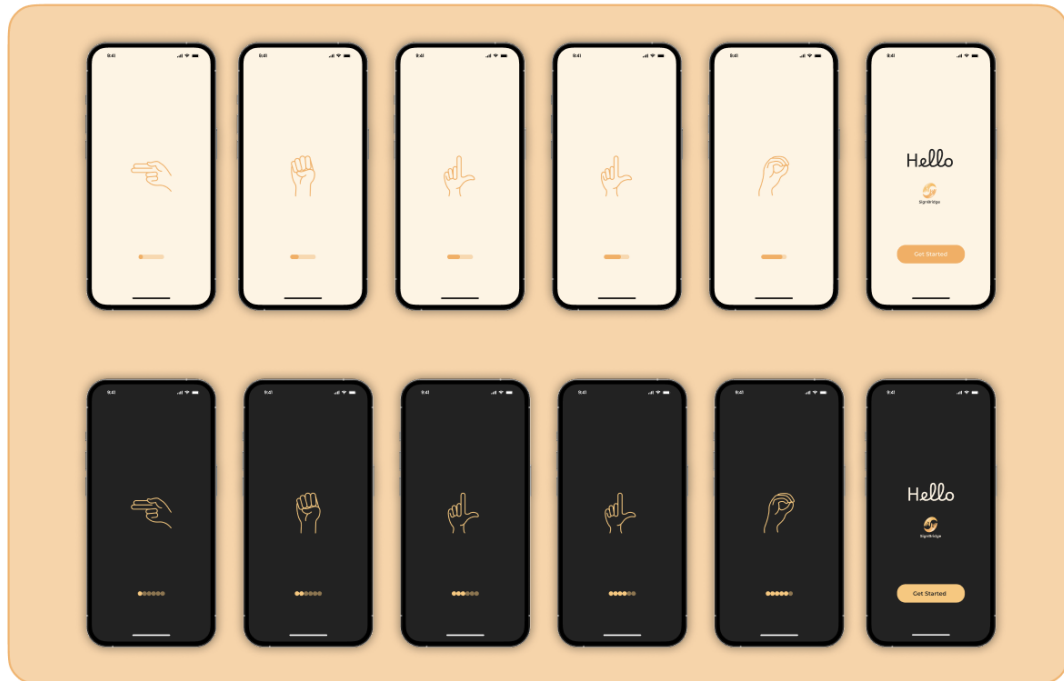


Figure 29: The main pages of SignBridge.

2 Development of SignBridge

2.1 Objective

The primary goal of SignBridge is to bridge the communication gap between Deaf and hearing individuals through an accessible and inclusive mobile application. Designed with a user-friendly interface, the app aims to support seamless interactions by recognizing the unique communication needs of the Deaf community. Special attention was given to the visual design, with a color scheme chosen based on research indicating that this audience responds positively to soft and engaging colors, enhancing usability and comfort.

2.2 Functionalities

SignBridge integrates a variety of:

- **Sign Recognition:** Users can record live videos or upload existing ones from the gallery to enable real-time Arabic Sign Language (ArSL) recognition and translation into readable text.
- **User Authentication:** Secure login and sign-up options provide personalized access and data protection.
- **Contact Us Page:** users have the ability to contact the administrator
- **Dark Mode Support:** To enhance visual comfort, especially in low-light environments, the app includes an optional dark mode feature. the figure 38 shows the use case of our mobile application , where the SignBridge's features are mentioned.

2.3 Technology Stack

2.3.1 Frontend (Flutter)

Flutter is an open-source UI software development kit created by Google. It enables developers to build natively compiled applications for mobile, web, and desktop from a single codebase. Flutter uses the Dart programming language and incorporates a rich set of customizable widgets, facilitating the creation of visually appealing and responsive user interfaces. [31]

2.3.2 Backend

2.3.2.1 Backend Server (Flask)

The backend of SignBridge responsible for video processing and sign recognition was developed using **Flask**, a lightweight Python web framework. Flask was used to create RESTful APIs that communicate with the Flutter app. It handles the reception of video files, preprocessing of frames, and inference using the trained deep learning model. Flask was chosen for its simplicity, scalability, and compatibility with machine learning workflows. the figure 30 illustrates the backend Workflow.

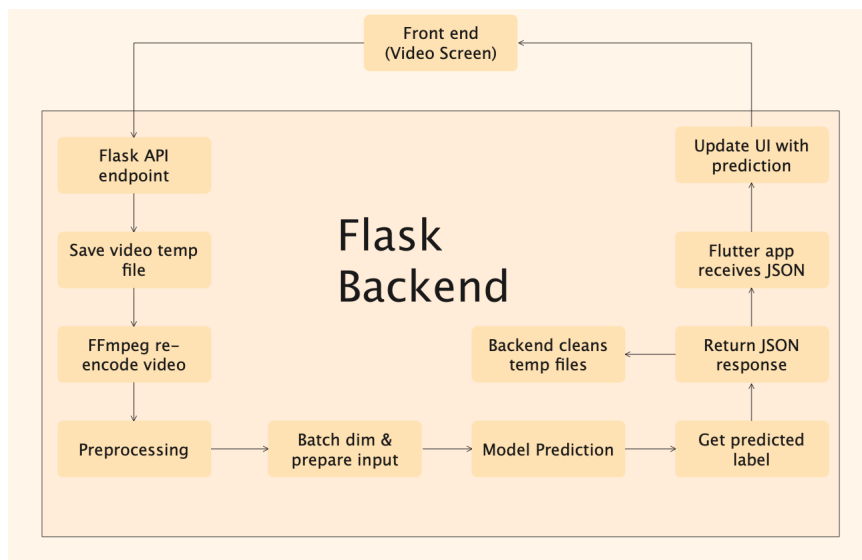


Figure 30: Backend Workflow of SignBridge.

2.3.2.2 HTTP Requests (API Communication)

The app utilizes Flutter's built-in **http** package to facilitate communication between the frontend and the Flask backend. This allows the app to send recorded or uploaded video files to the server and receive sign recognition predictions in response. The simplicity of the **http** package ensures efficient and straightforward API integration. The server returns the result in JSON format, from which the predicted label is extracted and displayed on the screen in a user-friendly manner. As shown in the figure 31

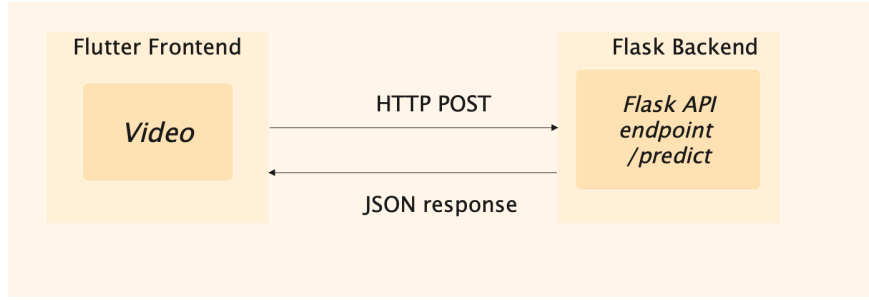


Figure 31: API Communication between the Flask Server and the Flutter App.

2.3.2.3 Our AI Model (.keras)

The core sign recognition functionality relies on a custom-trained deep learning model saved in the `.keras` format. This model receives preprocessed video frames and predicts the corresponding sign language gesture. The model uses a combination of convolutional and recurrent layers, MobileNetV2 + GRU to capture both spatial and temporal features. It is integrated into the Flask backend to provide real-time predictions. The figure 32 illustrates how the model is executed to generate real-time prediction .

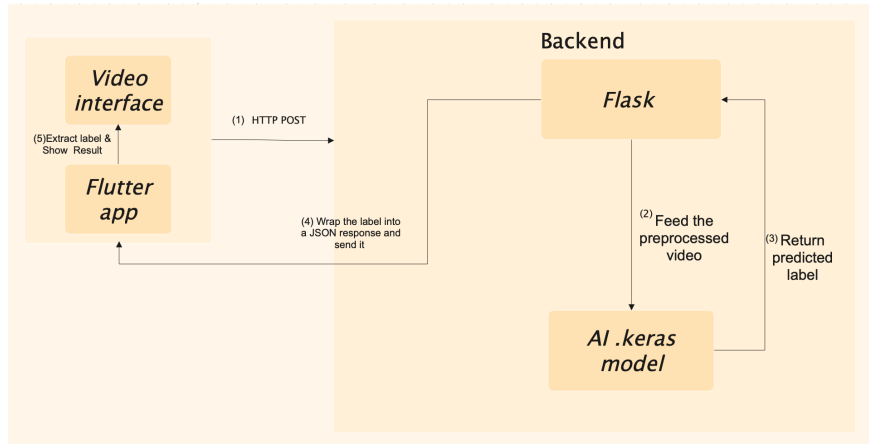


Figure 32: backend loads the trained model .

2.3.2.4 Firebase

Firebase [32] services were used for two critical functionalities:

- **Firebase Authentication:** Handles user registration and login using email/password and Google Sign-In methods. It ensures secure, personalized access to the app.

- **Firebase Firestore:** Used to store and manage messages submitted through the Contact Us page. When users send inquiries or feedback, the messages are saved in the Firestore database, allowing admins to review them via the Firebase console. The figure 33 illustrates the Contact Us Page.

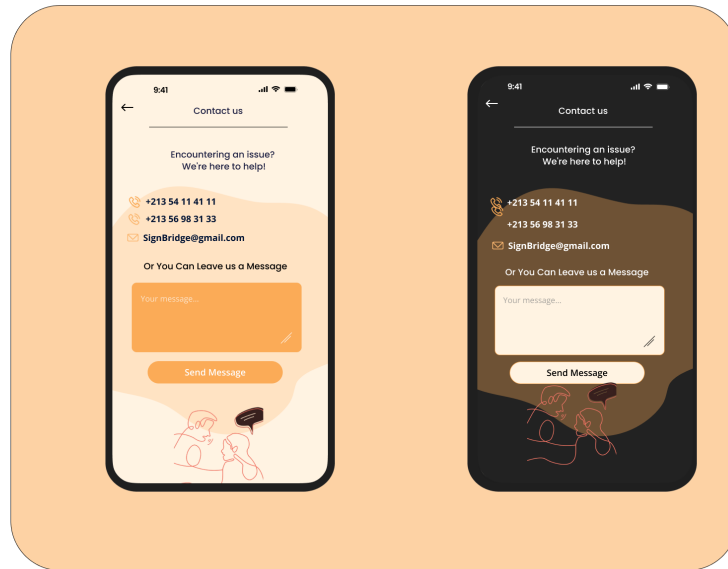


Figure 33: Contact Us Interface Using Firebase.

2.4 Architecture of SignBridge

SignBridge follows a modular and user-centered architecture to ensure clarity, maintainability, and scalability. The design is structured around real components used during the development process:

1. Frontend (Presentation Layer):

- Built using Flutter, the UI is divided into multiple screens such as onboarding, login/sign-up, main page, camera interface, and profile page.
- Custom widgets are used for UI consistency and modular design.
- The UI is responsive and supports both light and dark themes.

2. Backend (Flask Server):

- A Python-based Flask backend is used to process video input and run the trained sign language recognition model.
- The app sends video files through HTTP requests (POST) , and the backend returns the predicted label.

3. Firebase Integration:

- Firebase Authentication manages user login and registration using email and Google sign-in.
- The Contact Us messages are stored and accessed through Firebase, allowing administrators to view submitted messages.

4. Media Handling (Video Processing):

- Users can record a video or select one from the gallery using Flutter's camera and file picker package.
- The selected or recorded video is sent to the backend for processing.

5. Navigation:

- The app uses Flutter's navigation system for moving between screens such as Home page, camera page, profil page, common signs page.
- A bottom navigation bar helps users switch between main sections.

6. Theme Management:

- The app supports both light and dark themes.

2.5 Interface of SignBridge

2.5.1 Get Started Interface



Figure 34: Get Started Interface of SignBridge.

2.5.2 Sign In / Log In Interface

Then the user needs to Log In or Sign Up using Firebase Authentication .

2.5.3 Home Interface

Then after Logging In , a home interface will be displayed where he can consult the content section or directly go to the translation from sign to text section which is the main task of our app. As shown below in Figure 35

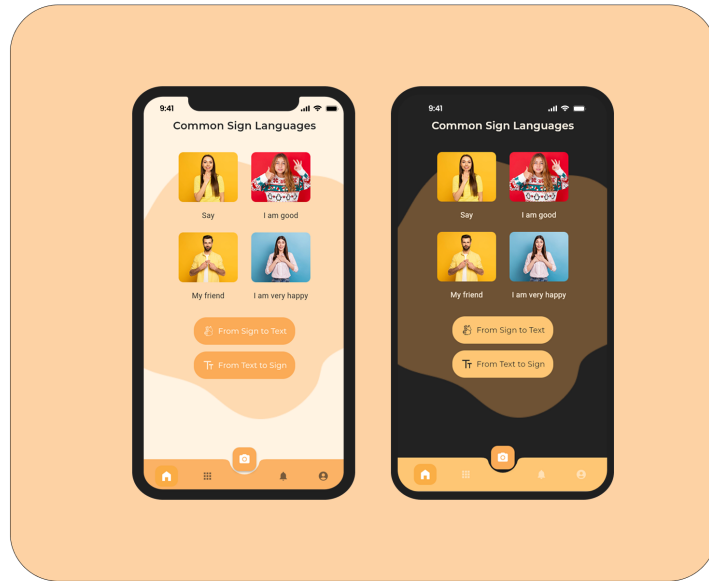


Figure 35: Home Interface SignBridge .

2.5.4 Recording / Uploading a Video Interface

After navigating to the translation from sign to text , the user will be facing two options, either to record his own video or upload from the gallery a pre-recorded video. As shown in Figure 36.

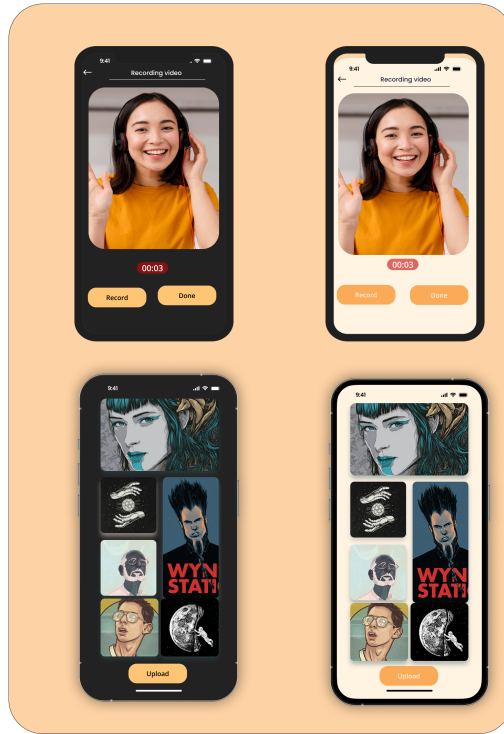


Figure 36: Recording / Uploading a Video Interface SignBridge.

2.5.5 Profile page

The profile page allows users to view their account information and manage settings such as theme preferences and language selection. It offers a simple, user-friendly interface for easy customization and navigation, As shown in Figure 37.

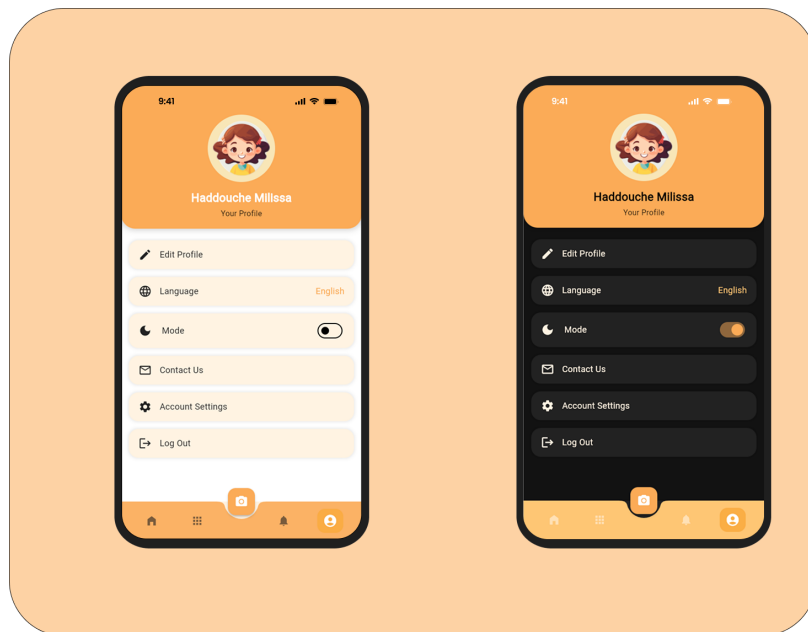


Figure 37: Profile Page of SignBridge.

3 UML Modeling of SignBridge

3.1 Use Case Diagram

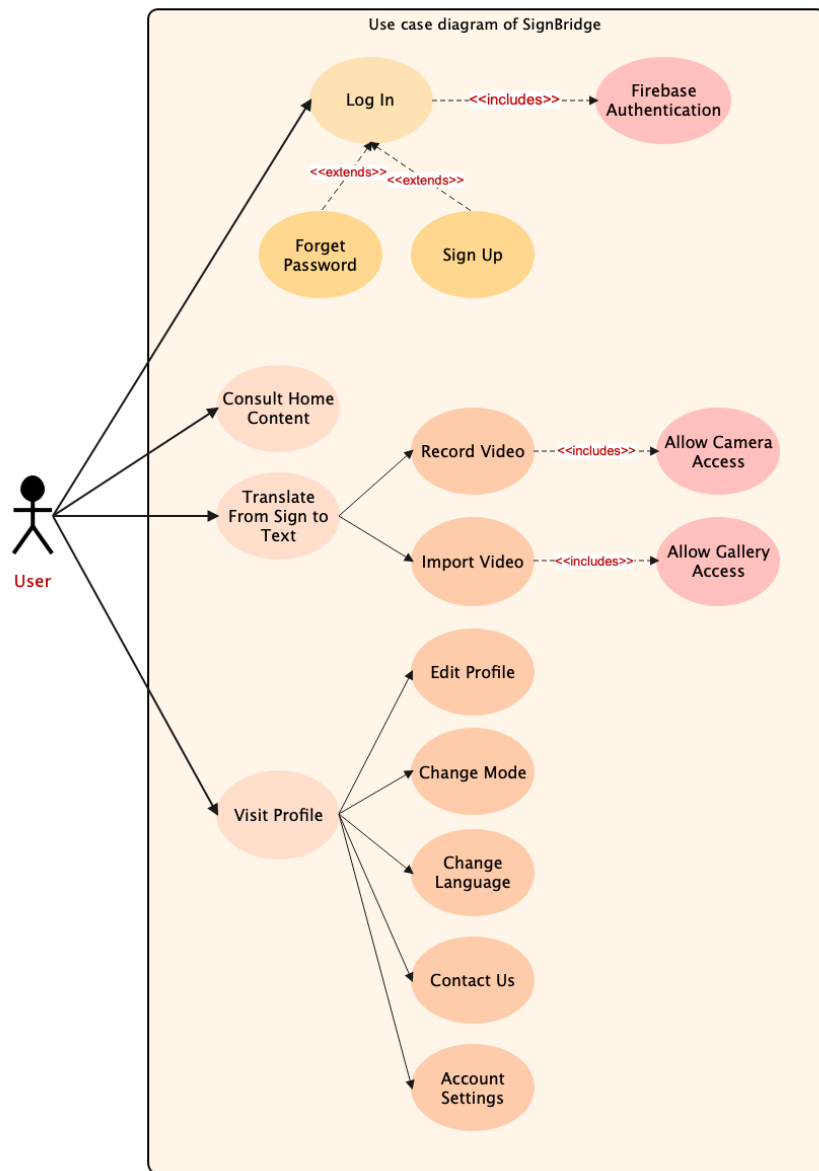


Figure 38: Use Case of SignBridge.

3.2 Activity Diagram

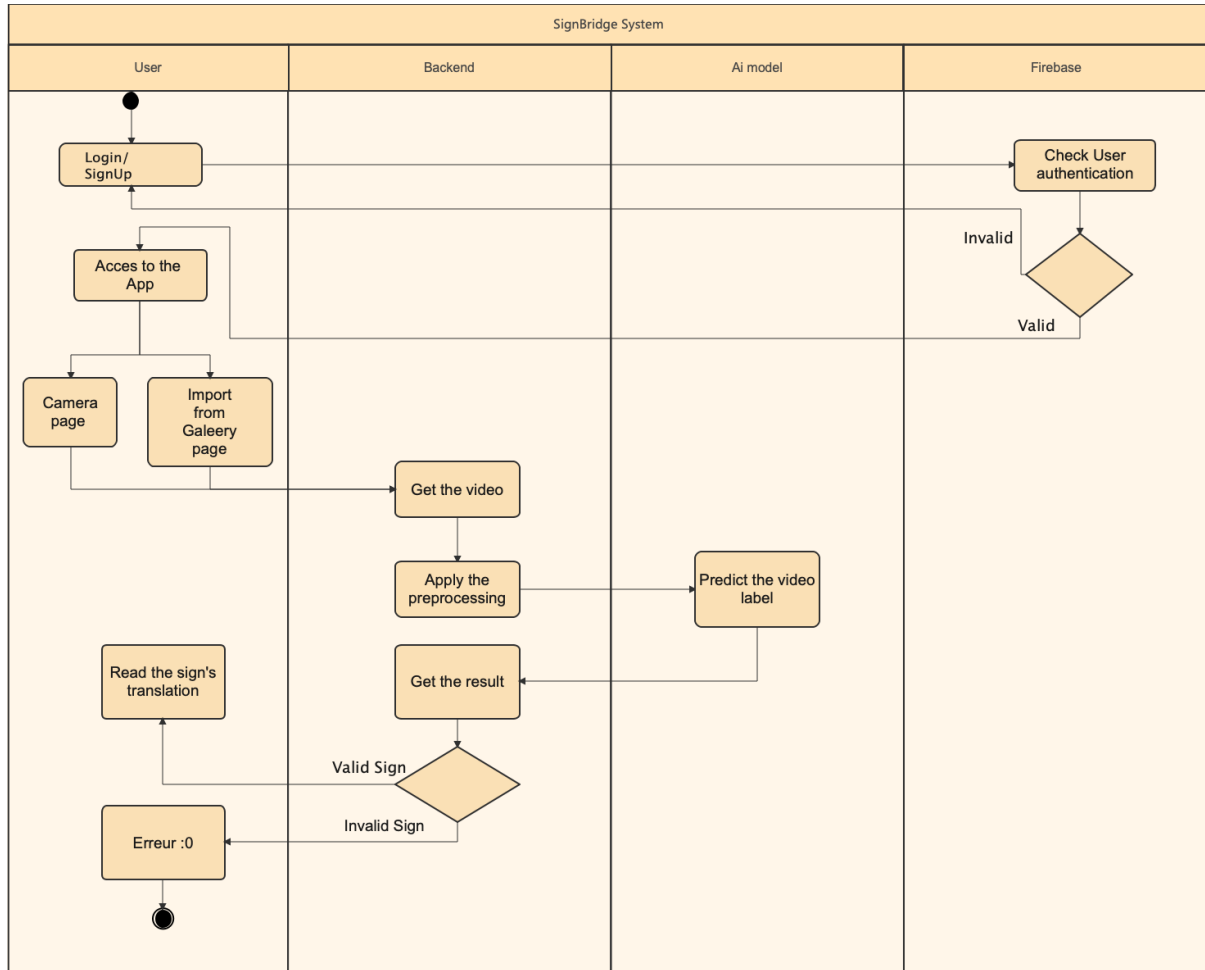


Figure 39: Activity diagram of SignBridge.

4 Deep Learning Framework

4.1 Dataset Preparation

We did not construct or collect a custom dataset for this study. Instead, we relied on publicly available resources hosted on Kaggle due to the absence of accessible Algerian Sign Language (ALGSL) datasets. Although we reached out to request access to ALGSL89 [28] the only dataset known to target Algerian Sign Language—we were informed that it is part of an ongoing startup and not yet publicly released.

Consequently, we based our experiments on two Egyptian Arabic Sign Language datasets:

- **ArSLVideoDataset:** Created by *Mohamed Saeed*, this dataset consists of 551 video samples across 12 Arabic sign classes, including “Alhamdulillah”, “Good morning”, “Salam aleikum”, and “Thanks”. The dataset is organized into training and validation sets, with an average of 40 and 6 samples per class, respectively. Each video is approximately 3 seconds long. [33] Classes distribution over dataset is shown in Figure 40.

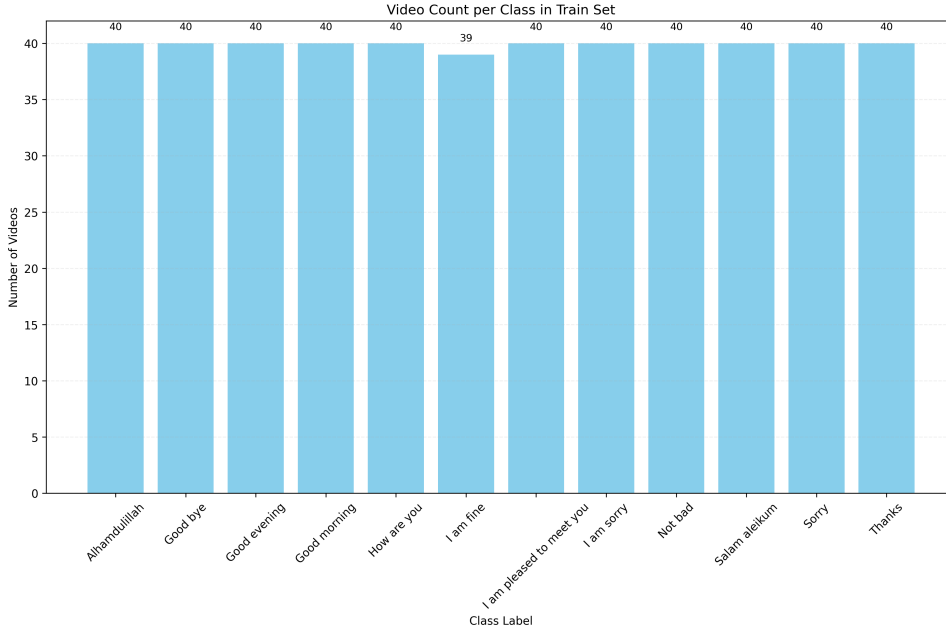


Figure 40: Video Files per Class in ArSL dataset Train Set.

- **ASL-20-Words-Dataset-v1:** Created by *Hossam Magdy Balaha*, this dataset

contains 8,498 video samples representing 20 different sign words such as “eat”, “father”, “love”, “sad”, and “worry”. The recordings were contributed by 72 volunteers aged between 20 and 24. Unlike the first dataset, this one is not pre-divided into train/test sets. The number of videos per class ranges from 366 to 452, with each video averaging around 3 seconds in duration. [28] [27] This dataset has a second version as well **arabic-sign-language-dataset** [34], because the first one that we tried to work with had a huge number of corrupted video files, and the difference between both versions is video files were compressed in the second one. Classes distribution is visualized below in Figure 41.

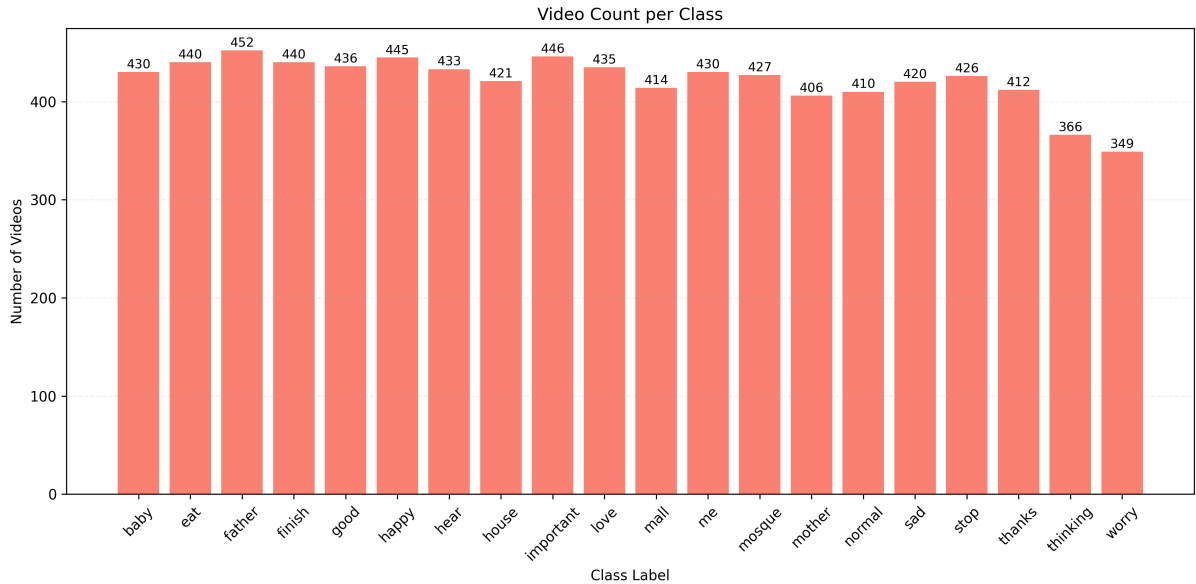


Figure 41: Video Files per class in arabic-sign-language-dataset.

4.2 Dataset Split

Since the **ArSLVideoDataset** was a small one and did not contain the test set we decided to work with **arabic-sign-language-dataset**, for the split phase, after a deep research we found that most studies go for 70/15/15 or mostly 80/10/10 ratio, so we opted since for 80% for train set, and 10% 10% for validation set and test set respectively since it is more commonly used and our dataset was not that large to work with only 70% for train set. The Figure 42 below visualizes the split that we performed.

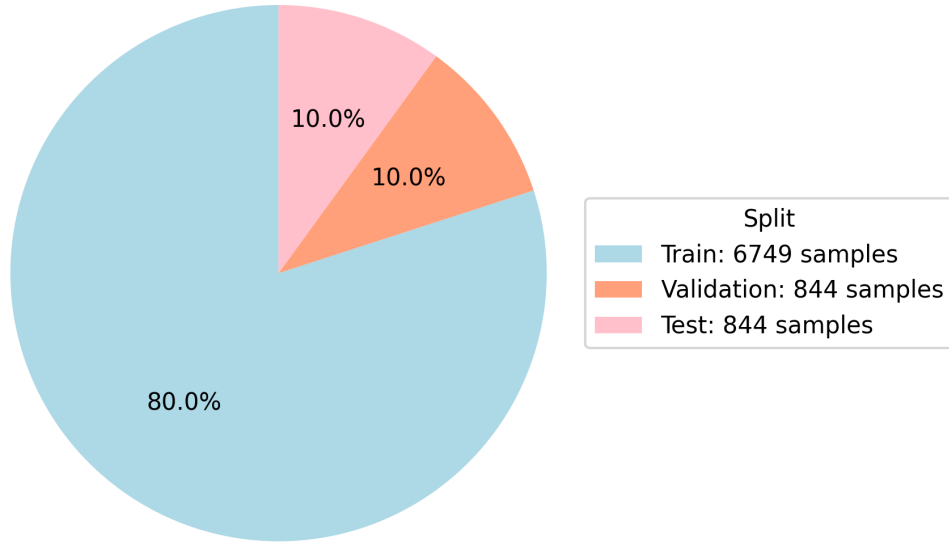


Figure 42: arabic-sign-language-dataset Split.

4.3 Preprocessing Techniques

In the preprocessing phase, raw video files were systematically converted into standardized and model-ready frame sequences. The implemented pipeline utilized `OpenCV` and `NumPy` to extract a fixed number of uniformly distributed frames from each video. Initially, 16 frames per video were extracted using linear interpolation across the total frame count. However, to capture more temporal information and improve model performance, the number of frames was later increased to 25.

Pipeline :

1. Frame Extraction

- Using `Numpy linspace` that generates 25 evenly spaced frame indices (as integers) for uniform sampling.

2. Color Conversion

- Using `OpenCV BGR2GRAY` that collapses RGB channels into a single luminance channel using standard weights.

3. Noise Reduction

- Using `OpenCV medianBlur` that applies a 5×5 median filter to remove impulse (“salt-and-pepper”) noise.

4. Contrast Enhancement

- Using `OpenCV equalizeHist` that performs histogram equalization, i.e. remaps pixel values via the image’s cumulative distribution function to enhance contrast.

5. Resizing

- Using `OpenCV resize` that rescales the image to 224×224 pixels using bilinear interpolation.

6. Normalization

- Dividing by 255.0 to convert integer pixel values to floats in $[0,1]$.

7. Channel Adjustment

- Using `Numpy newaxis` that adds a singleton channel dimension so a grayscale frame becomes shape $(H, W, 1)$.

8. Sequence Padding (if needed)

- Using `Numpy zeros` that creates a zero-filled array for any missing frames.
- Using `Numpy concatenate` that appends these zero frames along the time axis to reach a length of 25.

Output Specifications

- Tensor shape: $(25, 224, 224, 1)$
- Value range: $[0, 1]$
- Storage format: NPZ (compressed NumPy)

This preprocessing ensured that input data was consistent in shape, quality, and distribution, thereby improving the robustness and performance of the recognition model. The used approach is better explained in Figures [43](#) and [44](#).

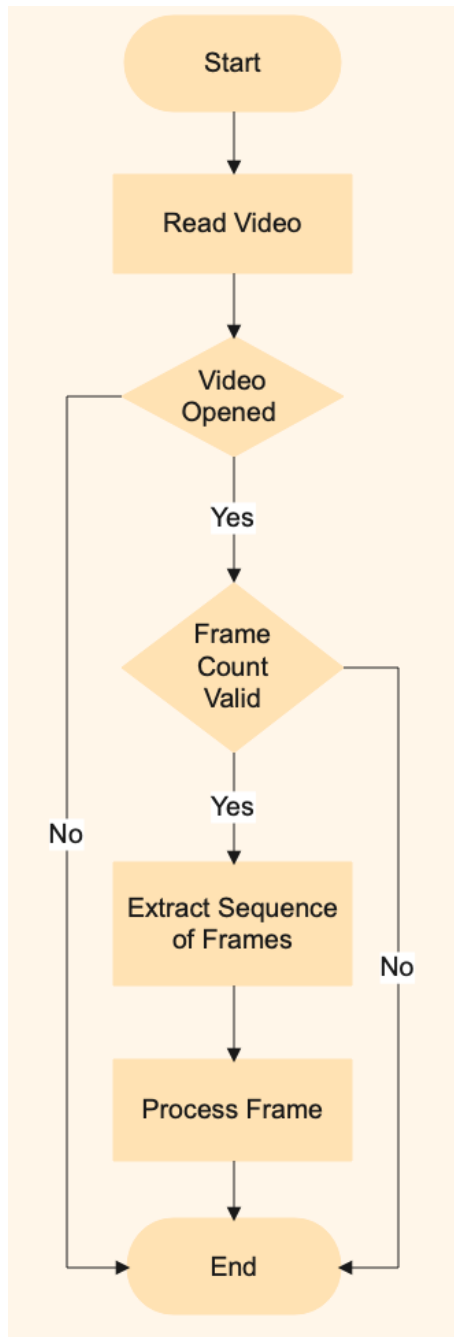


Figure 43: Frames Extraction Approach.

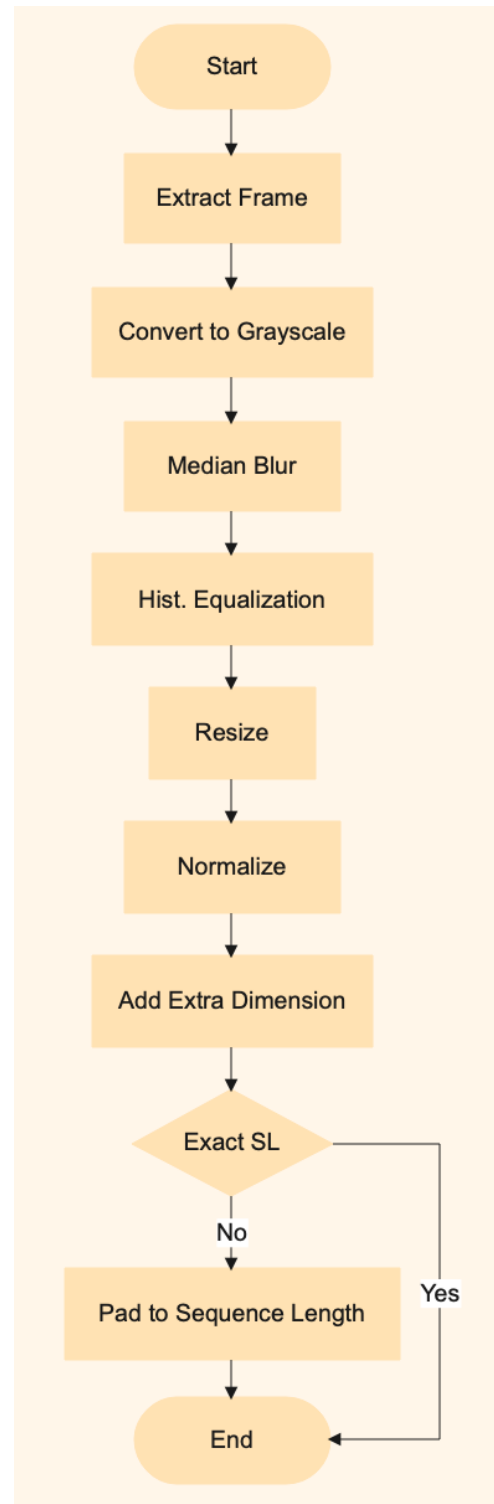


Figure 44: Frame Pre-processing Approach.

4.4 Architectures Used

After ensuring dataset availability and performing preprocessing phase, we moved to define the set of architectures of test on our mentioned datasets, here are the set of models that we tested:

1. MobileNetV2 + GRU (Hybrid)
2. I3D

4.4.1 MobileNetV2 + GRU

4.4.1.1 MobileNetV2

MobileNetV2 is a lightweight convolutional neural network (CNN) architecture developed by Google, designed specifically for mobile and embedded vision applications [35]. It builds upon the original MobileNet by introducing two key innovations: inverted residual blocks and linear bottlenecks. These architectural enhancements enable the model to achieve higher accuracy while maintaining low computational complexity.

The inverted residual structure allows for efficient feature reuse by connecting the bottleneck layers directly, facilitating better gradient flow and reducing the number of parameters. The use of linear bottlenecks helps in preserving information by avoiding non-linear transformations in narrow layers, which is crucial for maintaining representational power in low-dimensional spaces [35].

MobileNetV2 employs depthwise separable convolutions, which decompose standard convolutions into depthwise and pointwise convolutions. This decomposition significantly reduces the number of computations and model size without compromising performance [36].

Regarding input dimensions, MobileNetV2 typically expects images of size 224×224 pixels with three color channels, i.e., an input shape of $(224, 224, 3)$. However, it supports any input size greater than 32×32 , allowing flexibility based on specific application requirements [37].

Due to its compact size and efficiency, MobileNetV2 is particularly well-suited for real-time applications on mobile devices, such as sign language recognition. Its ability to deliver high accuracy with reduced latency makes it an ideal choice for deploying deep learning models in resource-constrained environments. As illustrated in the Figure 45.

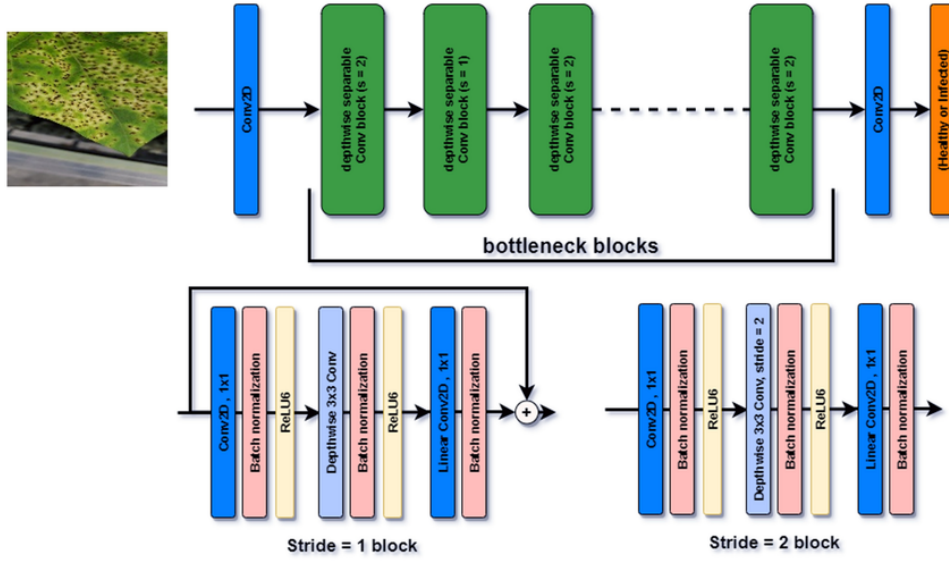


Figure 45: The Architecture of The MobileNetV2 with a Sample Input Image. [38]

4.4.1.2 GRU

Gated Recurrent Units (GRUs) are a type of Recurrent Neural Network (RNN) architecture introduced by Cho et al. in 2014 [39]. GRUs were designed to address the vanishing gradient problem commonly encountered in traditional RNNs, enabling the modeling of long-term dependencies in sequential data.

A GRU consists of two main gates: the update gate and the reset gate. The update gate determines how much of the past information needs to be passed along to the future, while the reset gate decides how much of the past information to forget. This gating mechanism allows GRUs to capture dependencies over varying time scales without the complexity of Long Short-Term Memory (LSTM) units, which have three gates and a separate memory cell [40].

Compared to LSTMs, GRUs have a simpler architecture with fewer parameters, leading to faster training times and reduced computational requirements. Despite their simplicity, GRUs have demonstrated comparable performance to LSTMs in various sequence modeling tasks, such as speech recognition and natural language processing [40].

In the context of mobile applications for sign language recognition, the efficiency and effectiveness of GRUs make them a suitable choice. Their ability to model temporal sequences with lower computational overhead aligns well with the resource constraints of mobile devices. As shown in Figure 46 below.

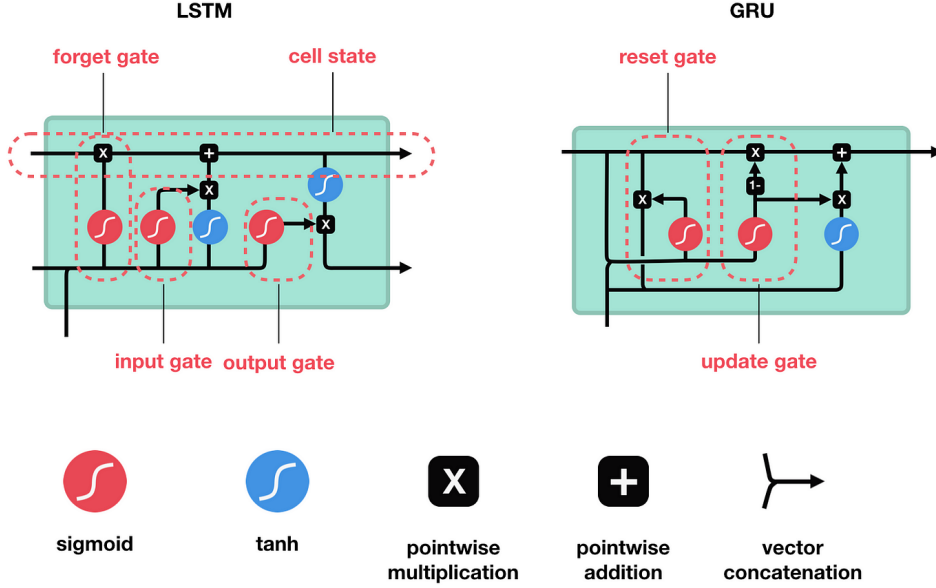


Figure 46: LSTM Vs GRU Architecture . [41]

4.4.1.3 Proposed Architecture

For both datasets we tested this hybrid customized architecture , **CNN (MobileNetV2)** for frame-level feature extraction and **stacked GRUs** for temporal sequence modeling. The input consists of grayscale video clips shaped as (25, 224, 224, 1), which are converted to pseudo-RGB via channel replication to leverage MobileNetV2’s pretrained ImageNet weights. Each frame is processed independently through a TimeDistributed MobileNetV2 (with frozen weights except the last 20 fine-tuned layers), followed by global average pooling and batch normalization to extract compact frame-level features. These features are then fed into two GRU layers (256 and 128 units) to capture sequential patterns, culminating in a classification head with a regularized dense layer (L2 penalty, 60% dropout) and a softmax output for 20-class prediction. The model balances efficiency (using lightweight MobileNetV2 and GRUs) and robustness (via batch norm, dropout, and partial fine-tuning), making it suitable for tasks like action recognition or sign language classification where both spatial and temporal features are critical. The architecture is visualized in the Figure 47.



Figure 47: MobileNetV2 + GRU Used Architecture .

Partial Fine-tuning is a deep learning technique where only a subset of a pre-trained model's parameters, typically specific layers are updated during training, while the remaining layers are kept frozen. This approach is particularly beneficial when computational resources are limited or when working with smaller datasets.

4.4.1.4 Hyperparameters and Formulas

1. GRU Update Rules Used in GRU layers (256 and 128 units), in order to capture sequential dependencies. The following are the gates equations [42]

$$\begin{aligned}
 r_t &= \sigma(W_r \cdot [h_{t-1}, x_t]) && \text{(Reset gate)} \\
 z_t &= \sigma(W_z \cdot [h_{t-1}, x_t]) && \text{(Update gate)} \\
 \tilde{h}_t &= \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t]) && \text{(Candidate hidden state)} \\
 h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t && \text{(New hidden state)}
 \end{aligned}$$

2. Batch Normalization Used in TimeDistributed after pooling, in order to normalize activations for stable and fast convergence. The following are its equivalent equa-

tions [43]

$$\begin{aligned}\mu_B &= \frac{1}{m} \sum_{i=1}^m x_i \\ \sigma_B^2 &= \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \\ \hat{x}_i &= \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \\ y_i &= \gamma \hat{x}_i + \beta\end{aligned}$$

3. Softmax Activation Used in final classification layer, in order to convert logits to probabilities for each class. The following is its equation [44]

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad K = 20$$

4. Cross-Entropy Loss Used in `sparse_categorical_crossentropy`, the purpose behind it is measuring prediction error. Commonly used in multi-class classification tasks where the true labels are provided as integers rather than one-hot encoded vectors.

5. L2 Regularization Used in Dense layer (`kernel_regularizer = L2(1e-3)`), in order to penalize large weights to prevent overfitting.

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{original}} + \lambda \sum_i w_i^2, \quad \lambda = 10^{-3}$$

This equation represents the addition of an L2 regularization term to the original loss function. The term $\lambda \sum_i w_i^2$ (sum of squares of weights) penalizes large weights, encouraging the model to maintain smaller weights, which can help prevent overfitting.

6. Dropout Used in Dropout layer (`rate = 0.6`), to randomly drop units to prevent co-adaptation.

$$y_i = \begin{cases} 0 & \text{with probability } p \\ \frac{x_i}{1-p} & \text{with probability } 1-p \end{cases}, \quad p = 0.6$$

This equation describes the Dropout technique, where each neuron's output x_i is set

to zero with probability p during training. If not dropped, the output is scaled by $\frac{1}{1-p}$ to maintain the expected value. This technique helps prevent overfitting and makes the network less reliant on specific neurons, encouraging it to learn redundant and more robust features.

7. Adam Optimization Used in optimizer with learning rate $\eta = 10^{-4}$, its purpose is adaptive learning rate with momentum (smooths out the updates by using the first moment — mean). The following is its equation [45]

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\ w_{t+1} &= w_t - \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \cdot \alpha \end{aligned}$$

Parameters:

$$\beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-6}, \quad \alpha = 10^{-4}$$

4.4.2 I3D

4.4.2.1 Inflated 3D ConvNet (I3D)

The Inflated 3D ConvNet (I3D), introduced by Carreira and Zisserman in 2017 [46], extends two-dimensional convolutional neural networks into the temporal domain, enabling the extraction of spatiotemporal features from video data. By inflating 2D convolutional filters and pooling kernels into three dimensions, I3D leverages pre-trained 2D models, such as Inception-v1, to process video inputs effectively.

A significant advantage of I3D over traditional 3D CNNs lies in its ability to utilize pre-trained weights from large-scale image datasets like ImageNet. This transfer learning approach facilitates more efficient training and improved performance, especially when video data is limited [46]. Additionally, I3D’s architecture incorporates a two-stream design, processing both RGB frames and optical flow to capture appearance and motion information, respectively. This dual-stream configuration enhances the model’s capability to recognize complex actions by combining spatial and temporal cues [46].

Empirical evaluations have demonstrated I3D’s superior performance on benchmark datasets. For instance, it achieved 98.0% accuracy on UCF-101 and 80.9% on HMDB-51, outperforming previous state-of-the-art models [46]. These results underscore I3D’s effectiveness in capturing nuanced spatiotemporal patterns, making it a robust choice for video-based action recognition tasks. As the following Figure 48 shows.

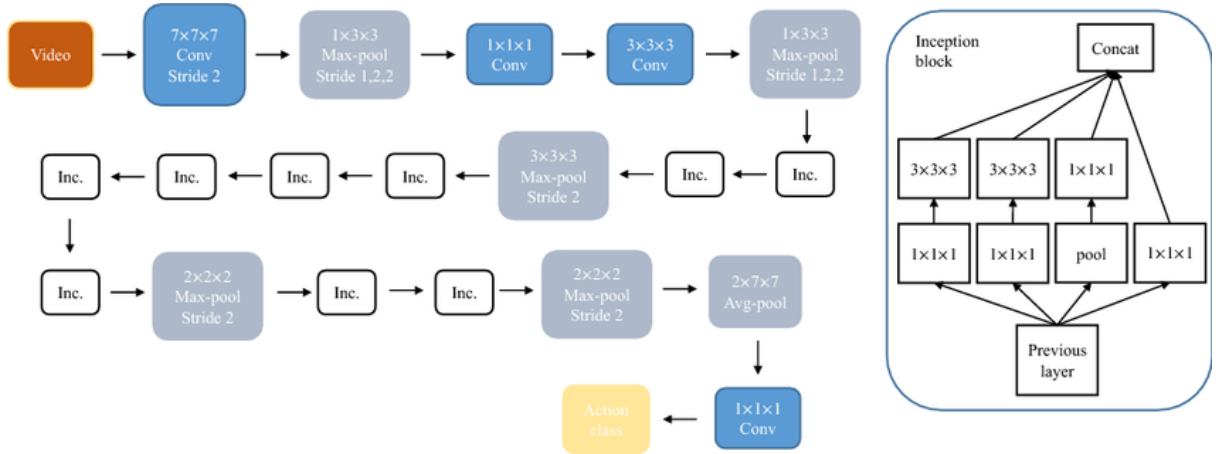


Figure 48: Structure of I3D (left) and its detailed inception submodule (right). [47]

4.4.2.2 Proposed Architecture

Subsequently, a three-dimensional convolutional neural network (3D CNN) architecture, namely Inflated 3D ConvNet (I3D), was evaluated on both datasets. I3D is specifically designed for video classification tasks, enabling the direct processing of spatiotemporal features capturing both spatial information (what appears in each frame) and temporal information (how things change across frames) from raw video clips. The term "Inflated" refers to the process of expanding 2D convolutional kernels into 3D ones. The model accepts input tensors of shape $(25, 224, 224, 1)$, corresponding to 25 grayscale frames with a resolution of 224×224 pixels.

The architecture initiates with a 3D convolutional layer employing a kernel size of $7 \times 7 \times 7$ to extract broad spatiotemporal patterns. This is followed by a hierarchical sequence of 3D convolutional layers with kernel sizes of $3 \times 3 \times 3$, utilizing an increasing number of filters (64, 128, and 256). Interleaved 3D max-pooling layers are applied to progressively downsample the spatial and temporal dimensions.

Subsequent to feature extraction, a GlobalAveragePooling3D layer condenses the output into a compact feature vector. To mitigate overfitting, a Dropout layer with a rate of 50% is incorporated. The final classification is performed using a Dense layer with a Softmax activation function, facilitating categorization into 20 distinct classes. The architecture was built as shown below in Figure 49

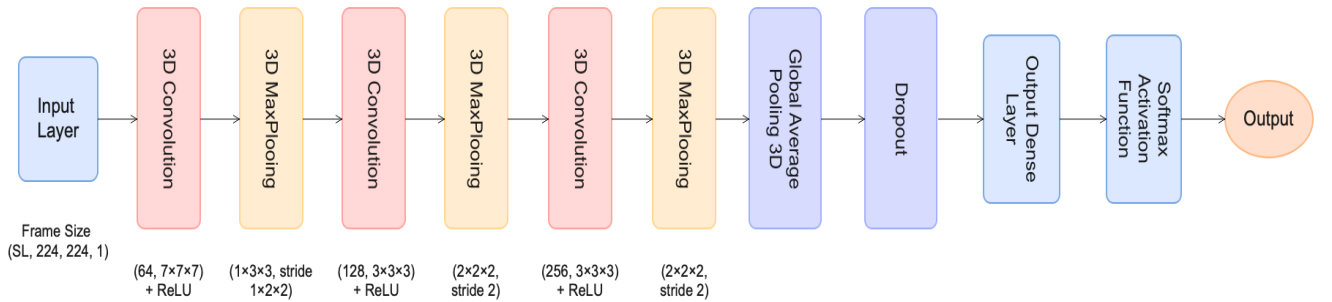


Figure 49: I3D Used Architecture .

4.4.2.3 Hyperparameters and Formulas

1. 3D Convolution Operation Used in all Conv3D layers, to extract spatiotemporal features from video tensors. This operation involves sliding the 3D kernel across the input volume in time and space, computing the weighted sum (dot product) at each position, and adding the bias term to produce the output feature map.

2. Max Pooling Used in all MaxPooling3D layers, in order to downsample 3D feature maps while retaining important activations. This operation slides a 3D window over the input tensor and computes the maximum value within that window for each position and channel. The result is a downsampled output tensor that retains the most prominent features, aiding in reducing computational complexity and mitigating overfitting.

3. Global Average Pooling (3D) Used in feature aggregation before classification to reduce spatiotemporal feature volume to a single feature vector. This operation computes the average value across the entire spatiotemporal volume for each channel, effectively summarizing the information contained in each feature map into a single scalar.

4. Dropout Used in dense pre-classification layer, to regularize training and prevent overfitting.

$$y_i = \begin{cases} 0 & \text{with probability } p \\ \frac{x_i}{1-p} & \text{with probability } 1-p \end{cases}, \quad p = 0.5$$

This equation describes the Dropout technique, where each neuron's output x_i is set to zero with probability p during training. If not dropped, the output is scaled by $\frac{1}{1-p}$ to maintain the expected value. This technique helps prevent overfitting and makes the network less reliant on specific neurons, encouraging it to learn redundant and more robust features.

5. Softmax Activation Used in final Dense layer, in order to convert logits to class probabilities. The following is its equation [44]

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad K = 20$$

6. Sparse Categorical Cross-Entropy Loss Used in training objective, to penalize incorrect predictions given integer-labeled classes. Commonly used in multi-class classification tasks where the true labels are provided as integers rather than one-hot encoded vectors.

7. Adam Optimizer Used in training with learning rate $\eta = 10^{-4}$, for efficient stochastic optimization with momentum (smooths out the updates by using the first moment — mean) and adaptive learning rates. The following is its equation [45]

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\ w_{t+1} &= w_t - \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \cdot \alpha \end{aligned}$$

Parameters:

$$\beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-6}, \quad \alpha = 10^{-4}$$

Chapter Conclusion

In this chapter, we detailed the end-to-end methodology of SignBridge, covering both the mobile application and AI model development. On the application side, we described our user-centered design process—from wireframes and UI/UX principles to state management, media processing, and Firebase integration—ensuring an accessible and extensible platform. On the AI side, we presented dataset selection, preprocessing pipelines (frame extraction, grayscale conversion, contrast enhancement, resizing, and padding), and two core architectures: an LRCN combining MobileNetV2 with GRUs, and a pure I3D 3D-CNN. We also discussed training protocols, hyperparameter tuning, and checkpointing on GPU resources. This comprehensive foundation sets the stage for the experimental evaluation in Chapter 3.

Chapter 3: Results & Discussion

Chapter Introduction

This chapter presents the empirical evaluation of our SignBridge system. We begin by outlining the experimental setup and metrics used for assessment. Next, we compare the performance of the LRCN (MobileNetV2 + GRU) and I3D models on our preprocessed datasets, including accuracy, precision, recall, and F1-score. Ablation studies on frame count, grayscale vs. RGB inputs, and hyperparameter variations are then analyzed to uncover their impacts on model behavior. Finally, we discuss the results in the context of real-world deployment, highlighting strengths, limitations, and avenues for future improvements.

1 Experimental Setup

1.1 Hardware Environment

All experiments were conducted on Kaggle’s hosted Jupyter environment, which provides the following hardware configuration:

- **GPU:** NVIDIA Tesla P100 with 16 GB of VRAM.
- **CPU:** Intel[®] Xeon[®] Processor @ 2.30 GHz (virtualized via Kaggle environment).
- **RAM:** 25 GB system memory.
- **Storage:** 75 GB SSD workspace with automatic persistence at `/kaggle/working`.

1.2 Software Environment

The software stack used throughout all experiments was as follows:

- **Operating System:** Linux (Ubuntu 18.04 LTS under the Kaggle container).
- **Python:** version 3.7.10.
- **Deep Learning Framework:** TensorFlow 2.11.0 (with Keras API).

- **CUDA & cuDNN:** CUDA 11.2, cuDNN 8.1.
- **Computer Vision:** OpenCV 4.6.0.
- **Numerical & Data Handling:**
 - NumPy 1.21.6.
 - Pandas 1.3.5
- **Machine Learning Utilities:** scikit-learn 1.0.2.
- **Visualization:** Matplotlib 3.5.1.
- **System & Process Management:**
 - **os** module (standard library).
 - **subprocess** module (standard library).
- **Utility Libraries:** tqdm 4.64.1 (progress bar utility).

All libraries were pre-installed in Kaggle’s default Python environment or installed via pip to ensure compatibility with the GPU runtime. The use of these specific versions ensured consistency and reproducibility across all experimental runs.

1.3 Test Device Specifications

- **Device Name:** OPPO Reno5
- **Processor:** Qualcomm[®] Snapdragon[™] 720G Octa-core
- **RAM:** 8.00 GB
- **Screen Size:** 6.4 in
- **Cameras:**
 - Front 44MP
 - Rear 64MP + 8MP + 2MP + 2MP
- **Android Version:** 13

2 Training Configuration

2.1 Dataset Splits & Preprocessing

For training and evaluation, we used the ”**Arabic Sign Language Dataset**” [34] publicly available on Kaggle. This dataset contains over 8,400 labeled video clips across 20 isolated Arabic sign classes, contributed by 72 participants. To facilitate robust model training and evaluation, the dataset was divided into three subsets following an 80/10/10 ratio as shown previously in Figure 42.

- **Training Set:** 6,749 video files.
- **Validation Set:** 844 video files.
- **Test Set:** 844 video files.

To prepare the videos for model input, we implemented a frame extraction and preprocessing pipeline. Each video was sampled into a fixed-length sequence of 25 frames, extracted evenly across the video’s duration. This decision was based on prior experimentation with shorter sequences (e.g., 16 frames), where 25 frames yielded better temporal coverage and performance.

The preprocessing process included **Grayscale conversion, Noise reduction, Histogram equalization, Resizing, Normalization, Channel adjustment** and **Padding**, as previously shown in Figures 43, 44.

2.2 Hyperparameters and Callbacks

To accommodate large video-based inputs without exceeding GPU memory, we employ a Keras `Sequence`-based data generator rather than loading all samples at once. The `VideoDataGenerator` streams batches of preprocessed frame-arrays from disk, each of shape `(batch_size, 25, 224, 224, 1)`, and yields corresponding integer labels. This on-the-fly batching allows us to keep the memory footprint constant regardless of dataset size.

2.2.1 Optimizer and Loss

We compile all models with the Adam optimizer and an initial learning rate of 1×10^{-4} . Sparse categorical cross-entropy is used as the training loss function, since our targets are integer-encoded class labels and the network’s final activation is softmax:

2.2.2 Training Callbacks

To accelerate convergence and guard against over- or under-fitting, three Keras callbacks are employed:

- **ReduceLROnPlateau** (`monitor='val_loss'`, `factor = 0.5`, `patience = 5`, `min_lr =  $1 \times 10^{-6}$`): automatically lowers the learning rate when validation loss stalls.
- **EarlyStopping** (`monitor='val_loss'`, `patience = 10`, `restore_best_weights=True`): halts training if no improvement is observed for 10 consecutive epochs and restores the best model, furthermore we tried training while monitoring validation precision (`monitor='val_precision'`, `patience = 10`, `restore_best_weights=True`)
- **ModelCheckpoint** (`save_best_only=True`, `monitor='val_loss'`): saves the model's weights at each epoch where validation loss reaches a new minimum.

2.2.3 Other Hyperparameters

- **Batch size:** varied between 4 and 8.
- **Number of epochs:** varied between 10 and 20.
- **Dropout rate:** 0.6 (in fully connected layers).
- **Weight regularization:** L2 with coefficient 1×10^{-3} on dense layers.

2.3 Evaluation Metrics

In order to comprehensively assess model performance, we report four core classification metrics: precision, recall, F1-score, and the confusion matrix, each highlighting a different aspect of the system's ability to correctly recognize sign-language gestures.

2.3.1 Precision

Precision measures the proportion of positive predictions that are actually correct. For each class c :

$$\text{Precision}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c},$$

where TP_c is the number of true positives for class c , and FP_c is the number of false positives assigned to c . High precision indicates that, when the model predicts a given sign, it is usually correct.

2.3.2 Recall

Recall (also called sensitivity) measures the proportion of actual positives that are correctly identified. For each class c :

$$\text{Recall}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c},$$

where FN_c is the number of false negatives for class c . High recall indicates that the model detects most instances of a sign.

2.3.3 F1-Score

The F1-score is the harmonic mean of precision and recall, providing a single measure that balances both:

$$\text{F1}_c = 2 \times \frac{\text{Precision}_c \times \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}.$$

A higher F1-score signifies a better trade-off between precision and recall for class c .

2.3.4 Confusion Matrix

The confusion matrix is an $N \times N$ table (for N classes) where the entry at row i , column j counts the number of examples whose true label is i but which the model predicted as j . It provides a detailed view of per-class errors and common misclassifications. As an example, the Figure 50 illustrated confusion matrix of the best model.

2.4 Quantitative Results

2.4.1 LRCN Vs I3D

2.4.1.1 Overall Metrics

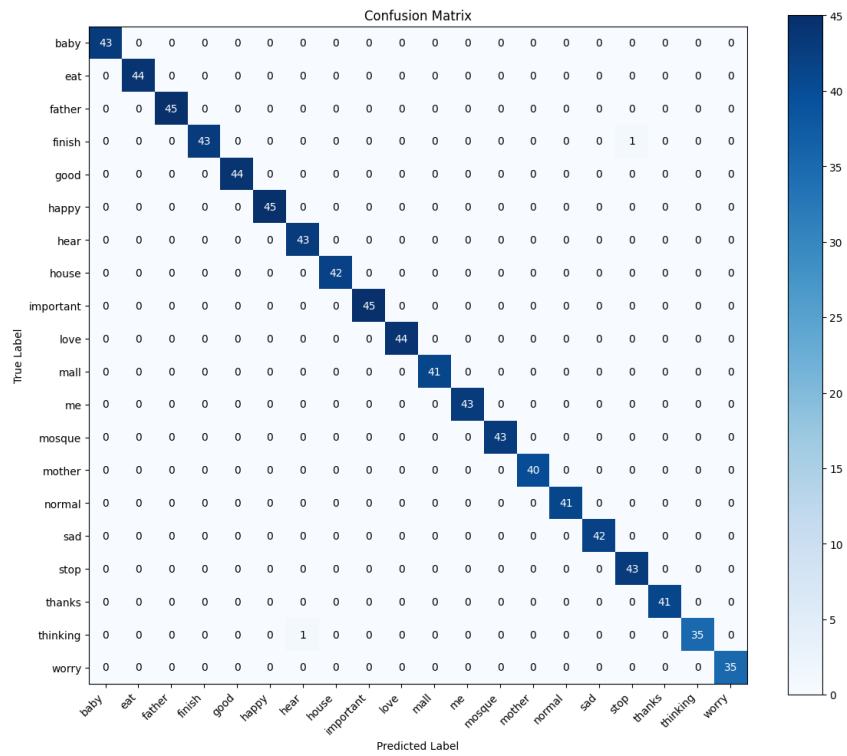


Figure 50: Confusion matrix for the best-performing model on the validation set.

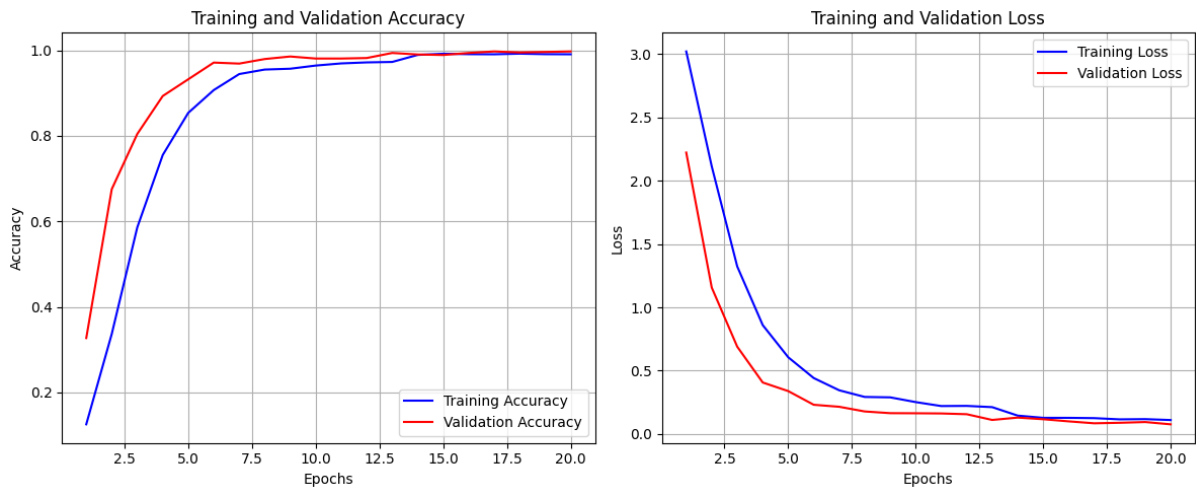


Figure 51: Training History of LRCN.

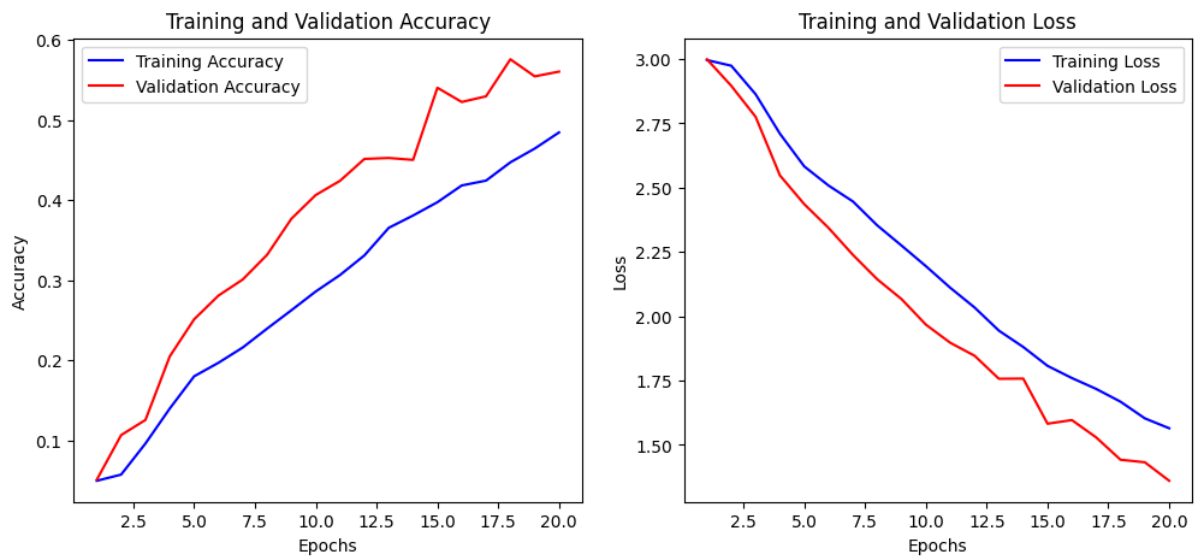


Figure 52: Training History of I3D.

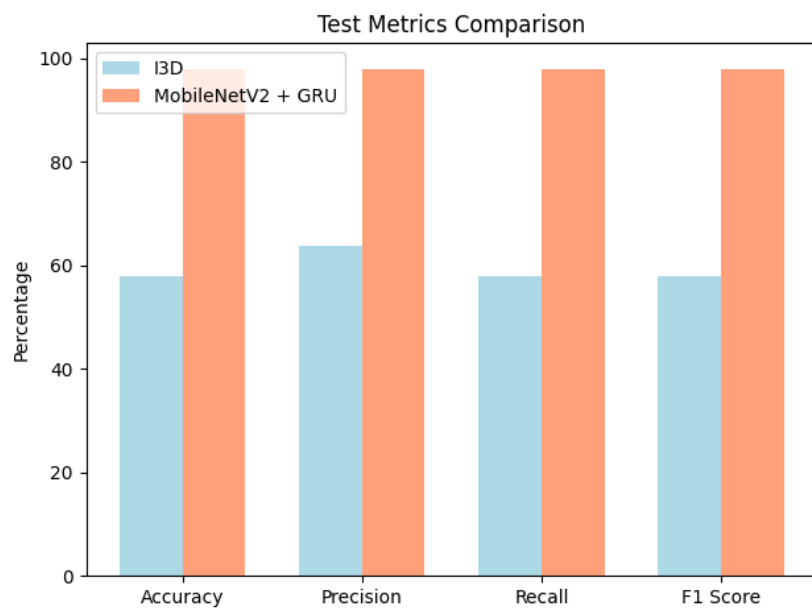


Figure 53: Metrics of Both Models on Test Set.

Epochs	Model	Train Acc (%)	Train Loss	Val Acc (%)	Val Loss
10	LRCN	94.8	0.31	97.3	0.19
	I3D	31.2	2.11	43.0	1.9
20	LRCN	98.0	0.10	99.5	0.08
	I3D	48.6	1.55	56.0	1.36

Table 3: Performance comparison of LRCN and I3D models over 10 and 20 epochs.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)
I3D	57.82	63.76	57.82	57.86
LRCN	99.76	99.77	99.75	99.76

Table 4: Performance comparison between I3D and LRCN on the test set

2.4.1.2 Per-Class Performance

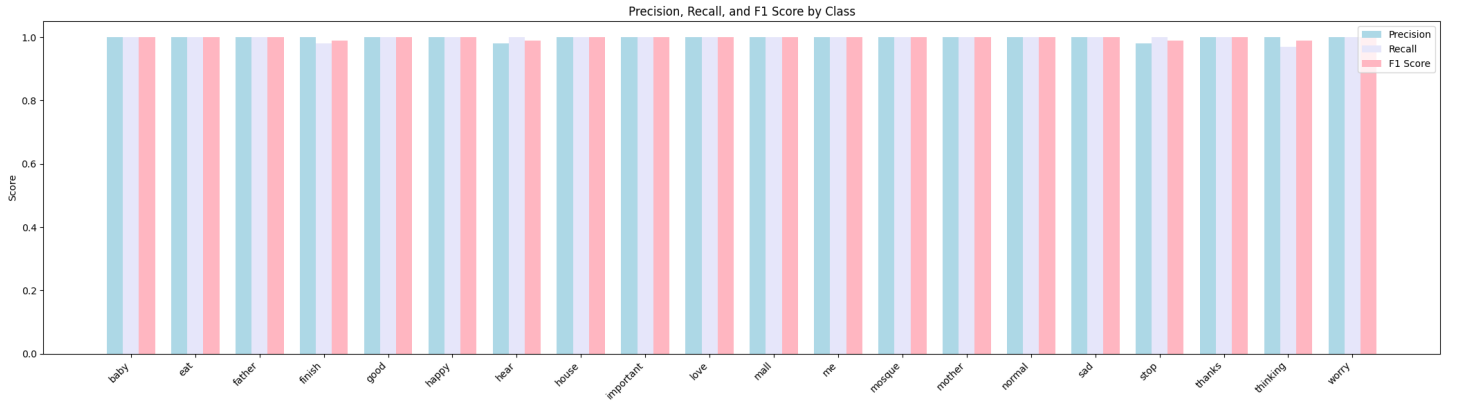


Figure 54: Per Class Metrics of LRCN on Test Set.

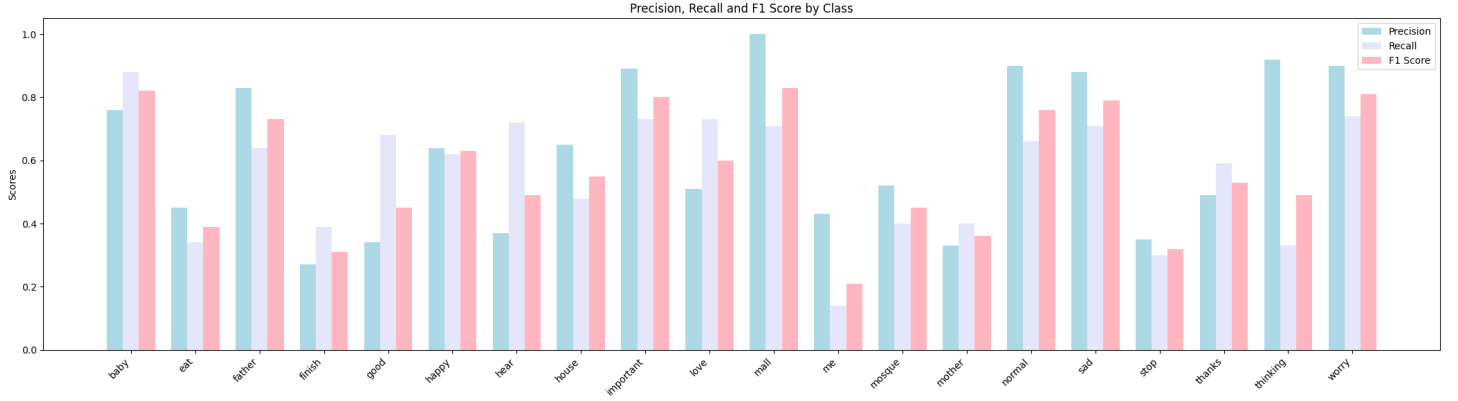


Figure 55: Per Class Metrics of I3D on Test Set.

Class	F1 Score (%)
Top 3 Best-handled Classes	
father	100.0
eat	100.0
baby	100.0
Bottom 3 Most Confused Classes	
finish	99.0
hear	99.0
stop	99.0

Table 5: LRCN per-class F1-score extremes on the test set.

Class	F1 Score (%)
Top 3 Best-handled Classes	
worry	81.0
baby	82.0
mall	85.0
Bottom 3 Most Confused Classes	
me	30.0
finish	31.0
stop	48.0

Table 6: I3D per-class F1-score extremes on the test set.

2.4.1.3 Confusion Matrix Comparison

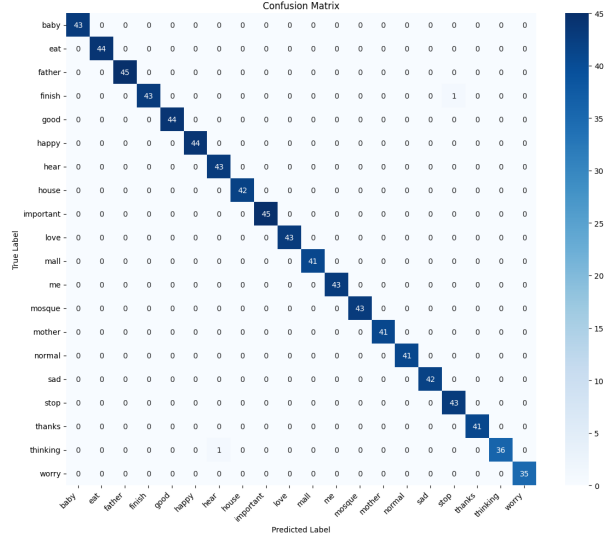


Figure 56: Confusion Matrix of LRCN on Test Set.

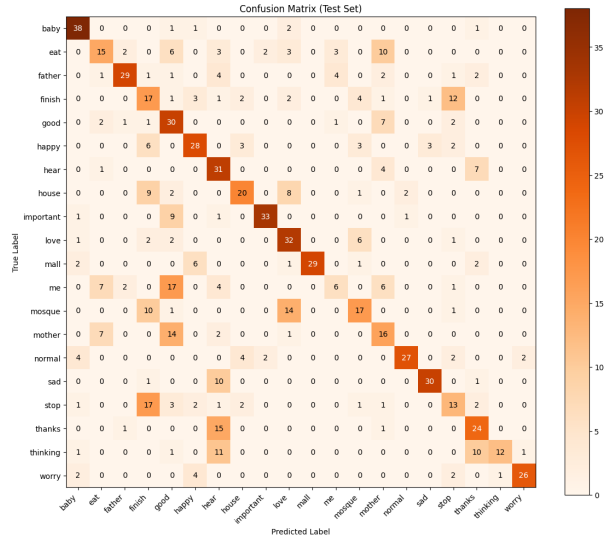


Figure 57: Confusion Matrix of I3D on Test Set.

2.4.1.4 Model Complexity & Inference Speed

Model	Params (M)	GFLOPs	Clip Latency (ms)	FPS
LRCN (MobileNetV2 + GRU)	3.61	15.32	170.9	5.9
I3D (Inflated 3D ConvNet)	1.13	34.92	68.6	14.6

Table 7: Comparison of model complexity and inference speed on the Kaggle P100 GPU.

2.4.2 Ablation Studies

Component	Finding	Implication
Frame Count (16)	Reduced performance due to limited temporal context	Longer sequences capture more comprehensive temporal features
Grayscale Input	Efficient for LRCN, potential performance drop for I3D	Grayscale is resource-friendly, RGB may be necessary for models like I3D
Partial Fine-Tuning	Fine-tuning last 20 layers improved results	Selective fine-tuning balances adaptation and retention of learned features

Table 8: Summary of Ablation Findings.

2.5 Qualitative Results



Figure 58: Frame from `important` Video File from The Test Set.

LRCN Prediction: `important` (99.94%)
I3D Prediction: `thinking` (24.24%)



Figure 59: Frame from `me` Video File from The Test Set.

LRCN Prediction: `me` (99.62%)
I3D Prediction: `father` (19.60%)



Figure 60: Frame from `mother` Recorded Video File.

LRCN Prediction: `mother` (88.99%)
I3D Prediction: `good` (26.90%)

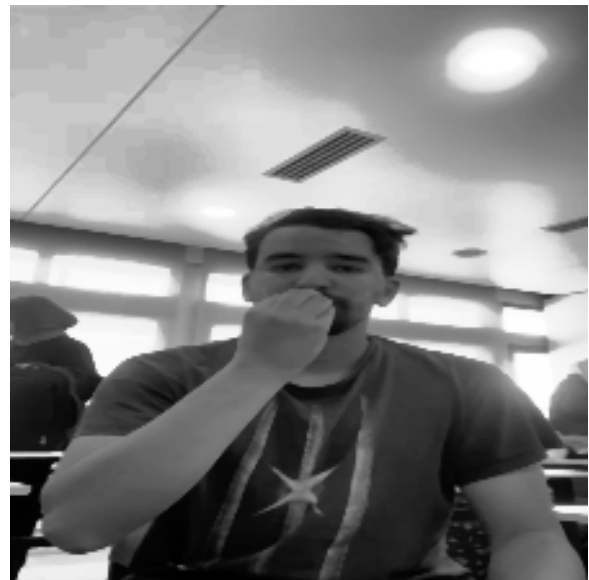


Figure 61: Frame from `eat` Recorded Video File.

LRCN Prediction: `eat` (97.95%)
I3D Prediction: `good` (21.32%)

2.6 Discussion

2.6.1 Comparison with Existing Work

To contextualize our performance, we compare our best LRCN results on the 20-word Arabic Sign Language dataset [34] (99.76% test accuracy) against the closest published study using the same data (Balaha *et al.*, 2023) [26], which reported 93.4% top-1 accuracy and 98.8% top-5 accuracy on UCF-101 but only 92% test accuracy on the 20-word Arabic Sign Language dataset. Table 9 summarizes this:

Model / Study	Dataset	Test Accuracy (%)
Balaha <i>et al.</i> (Vision-based CNN-RNN)	ArSL-20	92
Ours (LRCN: MobileNetV2+GRU)	ArSL-20	99.76

Table 9: Comparison of our LRCN performance against the best existing ArSL-20 result.

Due to several probable drawbacks:

- **Grayscale conversion & downsampling**
 - Completely discards color information that could aid hand/background separation.
 - Aggressive resizing risks losing fine detail (e.g. subtle finger movements).
- **Frame differencing & thresholding**
 - Fixed thresholding may drop frames with legitimate but subtle motion (slow or small gestures).
 - Amplifies camera noise, motion blur or lighting flicker, making motion masks unreliable.
- **Composite 3×3 frame aggregation**
 - Blurs the precise temporal order of frames, potentially confusing gestures distinguished by timing.
 - Enforces a rigid 30-frame length regardless of sign speed, compressing or stretching natural motion.

This direct comparison confirms that our architectural and pre-processing choices deliver state-of-the-art accuracy on this benchmark.

2.6.2 Interpretation of Results

Overall Training & Validation Performance

- **LRCN’s superior convergence:** After just 10 epochs, the LRCN reaches 94.8% training accuracy and already generalizes well (97.3% validation). By 20 epochs it achieves nearly perfect scores (98.0% train, 99.5% val) with low loss. Its recurrent GRU component effectively captures both short- and long-term temporal dependencies, enabling rapid learning of temporal patterns in sign sequences.
- **I3D’s slow start & underfitting:** In contrast, the pure 3D-CNN struggles early (31.2% train at 10 epochs) and only modestly improves by 20 epochs (48.6% train, 56.0% val). The heavier 3D convolutions demand far more data and training time to learn spatiotemporal filters, leading to underfitting under identical training schedules.

Test-Set Recognition Metrics

- **Robustness of temporal recurrence:** LRCN’s near-100% scores on all metrics demonstrate its ability to generalize reliably to unseen sign sequences. The GRU’s gating mechanisms preserve crucial temporal order information, which is essential when fine hand-motion distinctions depend on the flow of frames.
- **Limitations of 3D-only modeling:** Although I3D captures spatiotemporal features jointly, its relative underperformance ($\approx 58\%$ accuracy and F1) indicates that, with our dataset size and training budget, the model cannot fully learn discriminative 3D filters. Its higher propensity to underfit yields lower recall and F1, despite somewhat higher precision (suggesting many correct positives but missing numerous true instances).

Key Points:

1. Temporal recurrence outperforms pure 3D convolution on moderate-sized sign language datasets, thanks to GRU’s efficient sequence modeling.

2. I3D requires more data and epochs to reach comparable performance, making LRCN a more practical choice when resources are limited.
3. Hybrid CNN–RNN architectures strike the best balance between spatial feature extraction and temporal dependency learning for sign language recognition.

Per-Class Metric Analysis

- Figure 54 (LRCN) demonstrates uniformly high performance, with nearly every class at or above 0.90 and many at 1.00.
- Figure 55 (I3D) shows wide variation in all three metrics across classes, with many under 0.50, and only a few exceeding 0.80.

I3D – Best-handled Classes

- **mall:** P 0.97, R 0.76, F1 0.85
- **baby:** P 0.73, R 0.93, F1 0.82
- **worry:** P 0.84, R 0.77, F1 0.81

I3D – Most Confused Classes

- **me:** P 0.80, R 0.19, F1 0.30
- **finish:** P 0.28, R 0.34, F1 0.31
- **eat:** P 0.47, R 0.45, F1 0.46

Figure 62: Best vs. Worst Performing Classes for I3D on the Test Set

LRCN – Best-handled Classes

- Nearly all classes (**17/20**) at F1 = 1.00

LRCN – Most Confused Classes

- **finish:** P 1.00, R 0.98, F1 0.99
- **hear:** P 0.98, R 1.00, F1 0.99
- **stop:** P 0.98, R 1.00, F1 0.99

Figure 63: Best vs. Worst Performing Classes for LRCN on the Test Set

Interpretation:

- **LRCN clearly dominates:** almost every class is recognized flawlessly, indicating that the combination of frame-level MobileNetV2 features plus GRU sequence modeling is extremely effective on this isolated-word dataset.

- **I3D shows more variability:** while it excels on “mall,” “baby,” “worry,” its performance on short/ambiguous signs like “me,” “finish,” and “eat” suffers—likely due to the relatively shallow temporal context (25 frames) and the use of grayscale, which reduces appearance cues.
- **For deployment:** LRCN offers both high accuracy and consistency, whereas I3D may require richer input (e.g. RGB + optical flow) or longer clips to close its gap on the weaker classes.

2.6.3 Error Analysis

Most Frequent Misclassifications:

- **LRCN:** Only minor confusion for classes like:
 - ‘hear’ misclassified once as ‘thinking’.
 - ‘stop’ misclassified once as ‘finish’.
- **I3D:** Had more confusion for classes such as:
 - ‘finish’ was frequently misclassified as ‘stop’ (17 times).
 - ‘good’ was often predicted as ‘me’ (17 times) and ‘mother’ (14 times).
 - ‘thanks’ and ‘thinking’ had notable confusion, each misclassified as ‘hear’ and other nearby classes.

2.6.4 Limitations and Future Work

2.6.4.1 Challenges:

1. Large Dataset Size

- The combined dataset occupied over 36 GB of storage, exceeding the capacity of our local machines and complicating data transfer and storage management.

2. Hardware Constraints

- Local GPUs and RAM were insufficient to train deep models on full video datasets, forcing us to migrate to cloud-based notebooks.

- In Kaggle, we were limited to 30 GPU hours per week and 12 hours of continuous runtime, which restricted batch sizes, total epochs, and overall training time.

3. Online Environment Limitations

- Kaggle’s virtual environment imposed I/O and memory ceilings when streaming large video batches, preventing us from increasing the batch size beyond 4–8 samples.
- The 12-hour runtime cap required frequent checkpointing and restart, interrupting longer training experiments.

4. Model Complexity vs. Resources

- Our chosen architectures (LRCN and I3D) have substantial parameter counts and FLOPs, which strained available GPU memory and slowed training iterations on the limited hardware.

5. Data Availability and Quality

- Only two public Arabic sign-language datasets were available: one small (12 classes) of inconsistent video quality, and another (20 classes) containing corrupted files. We ultimately used the latter’s clean subset.
- Even the usable dataset covered just 20 words—far fewer than the hundreds in a real sign-language lexicon—limiting the scope and generalizability of our models.

6. Platform Heterogeneity

- In the application layer, targeting both web and mobile platforms introduced conflicting dependencies. Lacking OS-specific features on web builds, we had to simplify UI elements and delay certain native integrations (e.g. live camera feed handling) until a later native mobile rollout.

Despite these limitations, by carefully balancing batch size, sequence length, and model complexity, we were able to achieve robust proof-of-concept results and lay the groundwork for future scaling.

2.6.4.2 Future Scope:

Building on our current work, several avenues can be pursued to extend and strengthen the system:

1. Expanded, Custom Dataset

- Collect a Algerian, larger, in-house video corpus covering hundreds of sign classes.
- Ensure diversity in lighting, camera angles, signer demographics, and video quality to improve model robustness.

2. Advanced Model Architectures

- Experiment with deeper or hybrid networks (e.g., Transformer-based temporal modules, multi-stream 3D CNNs) to capture more complex sign nuances.
- Leverage semi-supervised and self-supervised learning to exploit unannotated video for richer feature representations.

3. Feature-Rich Application Enhancements

- Implement user history tracking, allowing individuals to review past translations and correct recognition errors.
- Integrate community-sourced content (e.g., user-submitted signs and definitions) to foster collaborative growth and dataset expansion.

4. Bidirectional Translation

- Extend the system to support bidirectional translation, convert written text back into sign animations using 3D avatar modeling or generative pose-sequence networks.
- Enable accessible two-way communication for both deaf and hearing users.

Chapter Conclusion

This chapter outlined our study key findings and interpretations, that the LRCN architecture combining MobileNetV2's spatial encoding with GRU sequence modeling rapidly

converged to 99.5% validation accuracy and achieved a 97.99% test accuracy with uniformly high per-class F1-scores. In contrast, the pure I3D model struggled under identical training conditions, peaking at 56% validation and 57.8% test accuracy, with notable confusion on finer gestures like “finish” and “me.” Ablation studies confirmed that longer frame sequences, grayscale preprocessing, and partial fine-tuning were critical to LRCN’s success. These findings underscore LRCN’s practicality for real-time, on-device sign-language recognition and provide a clear roadmap for enhancing data diversity and model complexity in future work.

Conclusion

This work set out to bridge the communication gap between deaf and hearing individuals by developing an always-accessible, lightweight mobile application for real-time Arabic sign-language recognition. We evaluated two deep-learning architectures—an LRCN model combining MobileNetV2 frame encoders with GRU sequence modeling, and a pure 3D-CNN I3D network—on a publicly available 20-word Arabic sign dataset, split 80/10/10 for training, validation, and testing. Our preprocessing pipeline extracted 25 evenly spaced grayscale frames per clip, applied median blurring and histogram equalization, and normalized pixel values; partial fine-tuning of the last 20 layers of MobileNetV2 and a dynamic learning rate (starting at 1×10^{-4}) further optimized training.

Quantitatively, LRCN achieved a stellar 99% validation accuracy and 97.99% test accuracy, misclassifying only three out of 844 test videos, with uniformly high per-class F1-scores. In contrast, I3D peaked at 56% validation and 57.82% test accuracy, showing strength on “mall,” “baby,” and “worry” ($F1 \geq 0.81$) but struggling on “me,” “finish,” and “eat” ($F1 \leq 0.46$). Ablation studies confirmed that increasing sequence length from 16 to 25 frames and employing grayscale inputs benefited LRCN, whereas I3D may perform better with RGB streams. Partial fine-tuning was essential for convergence.

Practically, LRCN’s 3.61 M parameters, 15.3 GFLOPs, and 170.9 ms clip latency (≈ 5.9 FPS) proved feasible for on-device use, whereas I3D’s lower parameter count (1.13 M) came at the cost of 34.9 GFLOPs and 68.6 ms latency (≈ 14.6 FPS), yet insufficient accuracy. User trials highlighted sensitivity to signer–camera distance and background clutter, underscoring the need for more varied training data.

Key limitations included the 36 GB dataset size, restricted local compute (pushing us to Kaggle’s 30 GPU-hour/week and 12-hour runtime limits), and a narrow 20-word vocabulary. Future work will focus on:

- Collecting a richer, multi-angle Algerian dataset covering hundreds of signs;
- Exploring deeper, hybrid architectures (e.g., Transformer modules, multi-stream 3D CNNs);
- Extending the app with history tracking, community content, and bidirectional text-to-sign translation via 3D avatar modeling;
- Further optimizing for real-world robustness across devices and environments.

References

- [1] *The Importance Of Learning Sign Language*. Accessed: 2025-04-28. 2021. URL: <https://yourdandi.co.uk/importance-learning-sign-language/>.
- [2] *Sign Language in Other Cultures*. Accessed: 2025-04-28. URL: <https://deafwebsites.com/sign-language-in-other-cultures/>.
- [3] *Discover just how diverse sign language is*. Accessed: 2025-04-28. 2021. URL: <https://truecolorsfestival.com/voice/sign-language-is-diverse#:~:text=There%20are%20more%20than%20300%2C%20use%20and%20understanding%20of%20signs..>
- [4] *Arabic Sign Language*. Accessed: 2025-04-28. 2025. URL: <https://www.qfi.org/resources/infographics/arabic-sign-language/>.
- [5] M.A. Abdel-Fattah. “Arabic Sign Language: A Perspective”. In: *The Journal of Deaf Studies and Deaf Education* 10.2 (2005). DOI: [10.1093/deafed/eni007](https://doi.org/10.1093/deafed/eni007). URL: <https://doi.org/10.1093/deafed/eni007>.
- [6] A. S. Elons and M. F. Tolba. “A Proposed Standardization for Arabic Sign Language Benchmark Database”. In: *Egyptian Journal of Language Engineering* 2.1 (2015). DOI: [10.21608/ejle.2015.60253](https://doi.org/10.21608/ejle.2015.60253). URL: <https://doi.org/10.21608/ejle.2015.60253>.
- [7] *Artificial Intelligence*. URL: https://www.sas.com/en_ca/insights/analytics/what-is-artificial-intelligence.html.
- [8] *Top 10 branches of Artificial Intelligence*. URL: <https://www.geeksforgeeks.org/top-10-branches-of-artificial-intelligence/>.
- [9] *What is Deep Learning?* URL: https://www.datacamp.com/tutorial/tutorial-deep-learning-tutorial?utm_source=google&utm_medium=paid_search&utm_campaignid=19589720821&utm_adgroupid=157156374951&utm_device=c&utm_keyword=&utm_matchtype=&utm_network=g&utm_adposition=&utm_creative=733936221293&utm_targetid=dsa-2218886984380&utm_loc_interest_ms=&utm_loc_physical_ms=9195866&utm_content=ps-other~emea-en~dsa~tofu~tutorial-machine-learning&accountid=9624585688&utm_campaign=230119_1-ps-other~dsa~tofu_2-b2c_3-emea_4-prc_5-na_6-na_7-le_8-pdsh-go_9-nb-e_10-na_11-na&gad_source=1&gad_campaignid=19589720821&gbraid=

OAAAAADQ9WsHH--0kpPS1LadyPV1ujDJsK&gclid=CjwKCAjwuIbBBhBvEiwAsNypvRo8jUvBNZFHkXnxxBa4JX54nC9odN4FSavOoCyOslahoC6pQQAvD_BwE.

- [10] *Deep Learning Definition*. URL: https://quantdare.com/wp-content/uploads/2019/06/deep_learning.png.
- [11] *Introduction to Convolution Neural Network*. URL: <https://www.geeksforgeeks.org/introduction-convolution-neural-network/>.
- [12] *What is convolutional neural network?* URL: https://media.licdn.com/dms/image/v2/D5612AQG0ui8XZUZJSA/article-cover_image-shrink_720_1280/article-cover_image-shrink_720_1280/0/1680532048475?e=2147483647&v=beta&t=5gZVHYNL2Vc2mK3iKrpK-FcpURIFdyaP4Vi38eeeZyM.
- [13] *What is a recurrent neural network?* URL: <https://www.ibm.com/think/topics/recurrent-neural-networks>.
- [14] *Gated Recurrent Unit*. URL: [https://www.sciencedirect.com/topics/computer-science/gated-recurrent-unit#:~:text=A%20Gated%20Recurrent%20Unit%20\(GRU,processing%20large%20amounts%20of%20data..](https://www.sciencedirect.com/topics/computer-science/gated-recurrent-unit#:~:text=A%20Gated%20Recurrent%20Unit%20(GRU,processing%20large%20amounts%20of%20data..)
- [15] *GRU*. URL: https://miro.medium.com/v2/resize:fit:423/1*i-yqUwAYTo2Mz-P1Q16MbA.png.
- [16] *MobilNetV2*. URL: https://paperswithcode.com/paper/mobilenetv2-inverted-residuals-and-linear?utm_source=chatgpt.com.
- [17] *MobileNetV2 Network Architecture*. URL: https://www.researchgate.net/figure/The-proposed-MobileNetV2-network-architecture_fig1_350152088.
- [18] *3D Convolutional Neural Networks*. URL: <https://www.sciencedirect.com/topics/computer-science/3d-convolutional-neural-networks>.
- [19] *3D Convolutional Neural Networks*. URL: <https://www.researchgate.net/publication/356066306/figure/fig1/AS:1088462463737884@1636521082423/3D-CNN-Architecture-The-CNN-consists-of-two-separate-parts-i-feature-extraction-and.png>.
- [20] Yingruo Fan et al. “Video-Based Emotion Recognition Using CNN-RNN and C3D Hybrid Networks”. In: *Proceedings of the 18th ACM International Conference on Multimodal Interaction*. ACM, 2016, pp. 445–450. DOI: [10.1145/2993148.2997632](https://doi.org/10.1145/2993148.2997632). URL: <https://doi.org/10.1145/2993148.2997632>.

- [21] Schalk Wilhelm Pienaar and Reza Malekian. “Human Activity Recognition Using LSTM-RNN Deep Neural Network Architecture”. In: *2019 IEEE 2nd Wireless Africa Conference (WAC)*. Pretoria, South Africa: IEEE, 2019. DOI: [10.1109/AFRICA.2019.8843403](https://doi.org/10.1109/AFRICA.2019.8843403). URL: <https://doi.org/10.1109/AFRICA.2019.8843403>.
- [22] Ruyi Zhang Xianyuan Wang Zhenjiang Miao and Shanshan Hao. “I3D-LSTM: A New Model for Human Action Recognition”. In: *OP Conf. Series: Materials Science and Engineering* 569 (2019), p. 032035. DOI: [10.1088/1757-899X/569/3/032035](https://doi.org/10.1088/1757-899X/569/3/032035). URL: <https://doi.org/10.1088/1757-899X/569/3/032035>.
- [23] Zhongwei Li et al. “I3D-Shufflenet Based Human Action Recognition”. In: *Algorithms* 13.11 (2020), p. 301. DOI: [10.3390/a13110301](https://doi.org/10.3390/a13110301). URL: <https://doi.org/10.3390/a13110301>.
- [24] Eaman Alharbi et al. “Real-time Detection and Monitoring of Public Littering Behavior Using Deep Learning for a Sustainable Environment”. In: *Scientific Reports* 15 (2025), p. 3000. DOI: [10.1038/s41598-024-77118-x](https://doi.org/10.1038/s41598-024-77118-x). URL: <https://doi.org/10.1038/s41598-024-77118-x>.
- [25] Krishan Kumarand Shikhar Sharma. “3D-CNN based dynamic American Sign Language recognition using multi-view videos”. In: *Multimedia Tools and Applications* 80.9 (2021), pp. 26319–26331. DOI: [10.1007/s11042-021-10768-5](https://doi.org/10.1007/s11042-021-10768-5). URL: <https://doi.org/10.1007/s11042-021-10768-5>.
- [26] Mostafa Magdy Balaha et al. “A vision-based deep learning approach for independent-users Arabic sign language interpretation”. In: *Multimedia Tools and Applications* 82 (2023), pp. 6807–6826. DOI: [10.1007/s11042-022-13423-9](https://doi.org/10.1007/s11042-022-13423-9). URL: <https://doi.org/10.1007/s11042-022-13423-9>.
- [27] *ASL 20-Words Dataset v1*. URL: <https://www.kaggle.com/datasets/hossammbalaha/asl-20-words-dataset-v1>.
- [28] Ahmed Kheldoun, Ilyes Kouar, and Errachid Ben Kouar. “ALGSL89: An Algerian Sign Language Dataset”. In: *International Journal of Management Information Systems and Computer Science* 7.2 (2023), pp. 128–141. DOI: [10.33461/uybisbbd.1339892](https://doi.org/10.33461/uybisbbd.1339892). URL: <https://doi.org/10.33461/uybisbbd.1339892>.
- [29] N. Alasmari and S. Asiri. “ASLDetect: Arabic sign language detection using ResNet and U-Net like component”. In: *Scientific Reports* 15 (2025), p. 18012. DOI: [10.1038/s41598-025-01588-w](https://doi.org/10.1038/s41598-025-01588-w). URL: <https://doi.org/10.1038/s41598-025-01588-w>.

- [30] *SignBridge UI/UX Design in Figma*. URL: <https://www.figma.com/design/px5Ru5L8XQ7qqu5P01pw5j/PFE?node-id=0-1&p=f&t=zG3t71531YUmUE0F-0>.
- [31] *Documentation of Flutter*. URL: <https://docs.flutter.dev/>.
- [32] *Documentation of Firebase*. URL: <https://firebase.google.com/docs/>.
- [33] *Arabic Sign Language Video Dataset*. URL: <https://www.kaggle.com/datasets/mohamedsaeed823/arslvideodataset/data>.
- [34] *Arabic sign language dataset*. URL: <https://www.kaggle.com/datasets/mahmoudmsaafan/arabic-sign-language-dataset>.
- [35] Mark Sandler et al. “MobileNetV2: Inverted Residuals and Linear Bottlenecks”. In: *arXiv preprint arXiv:1801.04381* (2018).
- [36] Andrew G Howard et al. “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications”. In: *arXiv preprint arXiv:1704.04861* (2017).
- [37] TensorFlow. *tf.keras.applications.MobileNetV2*. https://www.tensorflow.org/api_docs/python/tf/keras/applications/MobileNetV2. 2024.
- [38] *The architecture of the MobileNetV2 with a sample input image*. URL: https://www.researchgate.net/figure/The-architecture-of-the-MobileNetV2-with-a-sample-input-image-and-19-residual-bottleneck_fig8_369624227.
- [39] Kyunghyun Cho et al. “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation”. In: *arXiv preprint arXiv:1406.1078* (2014).
- [40] Junyoung Chung et al. “Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling”. In: *arXiv preprint arXiv:1412.3555* (2014).
- [41] *Architecture of LSTM Vs GRU*. URL: <https://pub.aimind.so/unlocking-sequential-intelligence-the-power-and-efficiency-of-gated-recurrent-units-in-deep-ef3c330b135f>.
- [42] *Gated Recurrent Unit Networks*. URL: <https://www.geeksforgeeks.org/gated-recurrent-unit-networks/>.
- [43] *What is Batch Normalization In Deep Learning?* URL: <https://www.geeksforgeeks.org/what-is-batch-normalization-in-deep-learning/>.
- [44] *Softmax Activation Function in Neural Networks*. URL: <https://www.geeksforgeeks.org/the-role-of-softmax-in-neural-networks-detailed-explanation-and-applications/>.

- [45] *What is Adam Optimizer?* URL: <https://www.geeksforgeeks.org/adam-optimizer/>.
- [46] Joao Carreira and Andrew Zisserman. “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 6299–6308. DOI: [10.1109/CVPR.2017.502](https://arxiv.org/abs/1705.07750). URL: <https://arxiv.org/abs/1705.07750>.
- [47] *Structure of I3D*. URL: https://www.researchgate.net/figure/Structure-of-I3D-left-and-its-detailed-inception-submodule-right_fig1_336976032.